Technical University of Applied Sciences Würzburg-Schweinfurt

NEXUSTEAM

Software Engineering Documentation Final Project Documentation

Requirements • Architecture • Implementation • UI • Testing • Project Management • Documentation

Team:

Sofiia Khyzhnychenko

Vladyslav Zaplitniy

Anna Kornet

Halil Hakan Karabay

Wurzburg 2026

TABLE OF CONTENTS

RESPONSIBLE PEOPLE

1. Project Overview — ALL
2. Requirements Management — Sofiia Khyzhnychenko
3. Architecture — Sofiia Khyzhnychenko
4. Backend and Client Implementation — Vladyslav Zaplitniy
5. User Interface — Anna Kornet
6. Testing — Halil Hakan Karabay
7. Project Management — ALL
8. Documentation — ALL
9. Conclusion and Future Work — ALL Appendix A — Diagrams

1. Project Overview

Introduction and Motivation

NexusTeam is a multi-platform communication system designed for real-time digital communication. The project combines instant messaging with file sharing, user presence, chat organization, voice messages and voice calls, AI image generation, and two client applications that share one backend.

The system is implemented as a client-server application. The web client(Nexus-Team/web) is a responsive browser application built with plain HTML, CSS, and JavaScript, served by Nginx and reachable from modern desktop and mobile browsers. It is the primary, universal client and requires no installation. The optional WPF desktop client (NexusTeam.Client, Windows-only, .NET 8) provides a native desktop experience. Both clients support realtime communication and live calls, but the media implementation differs: the web client uses browser WebRTC for audio/video calls and screen sharing, while the WPF client uses WebSocket call communication and NAudio-based raw audio capture/playback.

Running the project — for development or for evaluation — has a modest set of prerequisites: Docker Desktop (Windows/macOS) or Docker Engine plus Docker Compose v2 (Linux), with at least 8 GB of RAM allocated to Docker, since Oracle alone is a heavy container; the .NET 8 SDK, needed only if building outside Docker; and Git. Everything else — Oracle, the MongoDB shard cluster, Redis, the backend, and the web client — starts from one docker compose up -d --build in Nexus-Team/, with no manual database setup, because NexusTeam.DbSeeder provisions schemas, collections, and seed data automatically the first time the stack comes up. This is deliberate: a grader or a new contributor should be able to go from a fresh clone to a running, logged-in

session in one command and a few minutes, without hand-configuring three different database engines first.

Every image in the Compose file is published for both linux/amd64 and linux/arm64, so the same command line works unmodified on Windows, Intel macOS, Apple Silicon, and Linux — Docker Desktop simply runs the matching Linux image through its own Linux VM on non-Linux hosts. The first start pulls roughly 2 GB of images and takes two to four minutes; later starts, with images already cached, take about thirty seconds. The full set of services and the host ports they publish:

Service	**Purpose**	**Host port**
web	Nginx serving the SPA, reverse-proxying */api* and */ws*	8080
server	.NET 8 REST + WebSocket API	5251
oracle	Oracle Database 23ai Free — users	1530
mongos	MongoDB sharded cluster router — chats, messages	27018
redis	Cache, sessions, presence	6380
mongo-config, *mongo-shard1*, *mongo-shard2*	Cluster members	internal only
mongo-init, *mongo-router-init*, *db-seeder*	One-shot bootstrap jobs	—

Only web needs to be reachable to use the application day-to-day; the database ports are published mainly for convenience when running the .NET server or the WPF client directly on the host, outside their own containers, during development.

Both clients talk to the same ASP.NET Core 8 backend through REST APIs and a WebSocket channel. The main purpose of NexusTeam is to provide reliable real-time

communication while maintaining a modular and scalable software architecture, and to demonstrate modern software engineering practices: layered architecture, separation of concerns, polyglot persistence, containerization, JWT authentication, real-time communication, and structured project management.

The backend acts as the central component responsible for authentication, user management, chat management, messaging, file attachments, chat folders, user preferences, AI image generation, call signaling, and real-time communication. The system uses three specialized infrastructure components — Oracle Database, a sharded MongoDB cluster, and Redis. Docker Compose packages the whole stack (databases, backend, web client, and, in production, a TLS gateway and a coturn TURN relay for voice calls), and the application is deployed to a production Linux virtual machine reachable over a VPN.

Motivation. Modern communication platforms require more than simple text messaging. Users expect messages to be delivered in real time, files to be shared conveniently, online status to be visible, and conversations to remain available across devices. At the same time, communication systems need to address security, reliability, scalability, and maintainability.

The motivation behind NexusTeam was to develop a complete communication platform that combines these requirements in a single system, while giving the team an opportunity to apply software engineering concepts in a practical environment.

A particular focus was placed on separation of responsibilities. Instead of storing all information in one database or implementing all functionality in one layer, NexusTeam separates user-related relational data, communication-related document data, and temporary/high-frequency data:

- **Oracle Database** stores structured user-account data (the users table).

- **MongoDB**(deployed as a three-shard cluster with a config server and a mongos router) stores chats, messages, attachment metadata, chat folders, user preferences, generated-image records, call-history records, and user-device records. Attachment files themselves are written to the server file system by AttachmentService.
- **Redis** stores sessions, refresh tokens, presence, rate-limit counters, and other fast-access or temporary data.
- **ASP.NET Core** 8 provides the central backend application, exposing both REST and WebSocket endpoints.
- **WebSockets** provide realtime bidirectional communication for messaging, presence, typing indicators, and voice-call signaling.

Goals, Users, and Features

The main goal of the project is to develop a functional and extensible messaging platform capable of supporting both traditional and real-time communication. The primary objectives are:

- Provide secure user registration and authentication.
- Enable private and group-based communication.
- Provide real-time message exchange with delivery and read receipts.
- Support editing and deletion of messages.
- Support file and image attachment sharing.
- Provide user presence and status information.
- Allow users to organize conversations into chat folders.
- Support voice communication: recorded voice messages in both clients; live audio/video calls in the web client using browser WebRTC, ICE/TURN connectivity, and screen sharing; and live audio calls in the WPF client using

WebSocket call communication and NAudio. The shared backend handles call signaling, routing, and call history.

- Provide AI-assisted image generation inside a conversation.
- Run the whole stack reproducibly in Docker, and deploy it to a real server accessible outside the development machine.

Target Users.

- **Individual users** — register an account, authenticate, message other users, share files, and manage personal settings.
- **Group users** — participate in group chats with multiple participants.
- **Web users**— the responsive web client is the universal client: it works on modern desktop and mobile browsers and needs no installation. It provides messaging, attachments, folders, preferences, presence, voice messages, live audio/video calls, screen sharing, and image generation through the browser.
- **Windows desktop users**— can additionally run the WPF client for a native Windows experience. The WPF client supports the messaging workflow, voice messages, live audio calls, translation through DeepL, offline message queuing, and the same shared backend APIs/WebSocket protocol.

Main Features.

User Authentication. Registration and login are backed by JWT access tokens and Redis-backed refresh tokens; passwords are hashed with BCrypt. Authentication is centralized on the backend so both clients share one identity model.

Real-Time Messaging. A persistent WebSocket connection carries new messages, edits, deletions, typing indicators, delivery/read receipts, presence changes, and reactions, so neither client has to poll for updates.

Chats, Groups, and Folders. Users can create private and group chats and organize them into folders.

Attachments. Files, images, audio, and code snippets can be attached to messages (up to 100 MB per file); images get thumbnails and code attachments get language-aware preview.

Voice. Both clients can record and send voice messages. The web client additionally supports live audio/video calls using browser WebRTC, ICE servers and screen sharing. The WPF client supports live audio calls using WebSocket call messages and NAudio capture/playback. The server routes call signaling and WPF audio data and records call history; coturn provides TURN relay support for browser WebRTC when required.

AI Image Generation. Users can generate images from a text prompt directly inside a chat; generated images are stored as first-class attachments.

Translation. The WPF client provides on-demand message translation through a dedicated DeepL API service. The current web client does not implement the translation workflow.

Presence and Status. Online/away/offline/DND status and last-seen information are tracked in Redis and pushed to clients over WebSocket.

Themes and Preferences. Light/dark/auto theme, notification, privacy, and chat-display preferences are stored per user in MongoDB.

NexusTeam combines core messaging functions with supporting features such as attachments, folders, preferences, voice communication, AI image generation, and translation. Authentication, messaging, and presence form the core communication workflow and are covered by the main test suites in Chapter 6. The additional features extend this workflow without changing the overall client-server architecture.

Technology, Structure, and Scope

Layer	Technology	Purpose
Primary client	HTML / CSS / JavaScript + Nginx	Universal responsive web UI, browser WebRTC calls and screen sharing
Optional client	WPF (.NET 8, Windows) + CommunityToolkit.Mvvm	Desktop UI, live audio calls, translation, offline message queue
Server	ASP.NET Core 8	REST API and WebSocket host
User database	Oracle Database 23ai	User accounts and structured user data
Message database	MongoDB, sharded cluster	Chats, messages, attachment metadata, preferences, folders, generated images, call history, device state
Cache / ephemeral store	Redis	Sessions, refresh tokens, presence, rate limiting, cache
Auth	JWT + refresh tokens	Stateless authentication
Password hashing	BCrypt	Credential storage
Validation	FluentValidation	Request validation
Logging	Serilog	Structured logging
Serialization	System.Text.Json (source-generated)	High-performance JSON
Voice calls	WebRTC + WebSocket signaling +	Browser audio/video calls and

	coturn (web); WebSocket + NAudio (WPF)	screen sharing; WPF live audio
Voice messages (web)	MediaRecorder API (web) + NAudio (WPF)	Client-side voice-note recording
Containerization	Docker / Docker Compose	Reproducible dev and production environment
Deployment	*deploy.sh* over SSH/VPN, Let's Encrypt (Certbot), Nginx TLS gateway	Production release to a Linux VM

System Overview. At a high level, NexusTeam follows a client-server architecture. Clients are responsible for presentation and user interaction; the backend provides centralized business logic, authentication, data access, and realtime communication.

The two clients are:

- the **web client,** the primary universal browser-based client (HTML/CSS/JS, served by Nginx);
- the **WPF desktop client**, an optional Windows-only client used for development and voice calls.

Both communicate with the same backend. REST is used for request-response operations; WebSocket is used for realtime events, message operations, presence, typing, and call communication. The backend itself is layered: controllers receive requests, services contain business logic, repositories provide data access, and three specialized data stores hold the corresponding data.

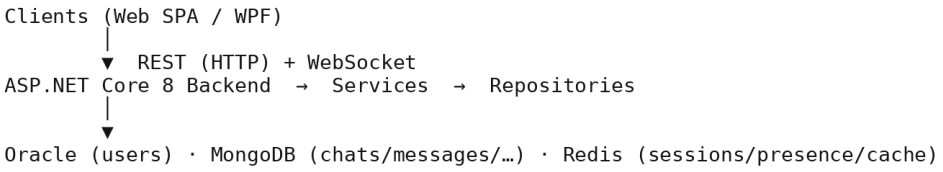


Figure 1 — System Overview — Client/Server/Data Flow.

Neither client accesses Oracle, MongoDB, or Redis directly — every data operation is mediated by the backend.

Project Structure.

- **NexusTeam.Server** — the ASP.NET Core backend: controllers, services, repositories, middleware, configuration, validation, authentication, and WebSocket handling.
- **NexusTeam.Client** — the optional WPF desktop application, structured with the MVVM pattern (views, view models, services, infrastructure).
- web/ — the primary web client: static HTML/CSS/JS served by Nginx, proxying /api and /ws to the backend.
- **NexusTeam.Shared** — models, DTOs, enums, exceptions, and WebSocket contracts shared between server and both clients, so they cannot drift out of sync.
- **NexusTeam.DbSeeder** — a one-shot container that initializes database schemas/collections and seeds demo data (including four demo accounts).
- **deploy**/, deploy.sh, docker-compose.yaml — deployment tooling: the Nginx/Certbot TLS gateway configuration and the script that ships a committed revision to the production VM.

Project Scope. The scope covers the development of the communication platform, its two clients, backend infrastructure, three databases, security mechanisms, containerized deployment, a real production deployment to a virtual machine, and supporting documentation.

As an academic software engineering project, some components remain intentionally simplified for the current milestone: the demo-account seeder recreates fixed demo passwords on every startup (fine for a VPN-only demo, but explicitly flagged as unsuitable for public exposure without changes — see §3.36 and §8.17), automated test coverage is still growing, and several security-hardening items are tracked as completed milestones rather than a finished backlog (see §8 and the project TODO.md).

Document Structure.

- **Chapter 1 – Project Overview:** introduces NexusTeam, its motivation, goals, target users, main features, technology stack, and scope.
- **Chapter 2 – Requirements Management:** functional and non-functional requirements and how they shaped development.
- **Chapter 3 – Architecture:** overall system architecture, application layers, communication mechanisms, databases, infrastructure, deployment, and security architecture, with diagrams.
- **Chapter 4 – Backend and Client Implementation: how** the main backend and client features were implemented.
- **Chapter 5 – User Interface:** design and implementation of both clients' interfaces.
- **Chapter 6 – Testing:** testing strategy, test cases, results, and limitations.
- **Chapter 7 – Project Management:** team organization, task distribution, workflow, version control, coordination.
- **Chapter 8 – Documentation:** documentation produced for developers, deployment, architecture, security, and maintenance.
- **Chapter 9 – Conclusion and Future Work:** achievements, evaluation, and planned improvements.

2. Requirements Management

Introduction, Scope, and Stakeholders

Requirements management defines what NexusTeam should provide to its users, which technical and quality characteristics the system should have, and which constraints have to be considered during development. Requirements were considered from both a functional perspective (what the system does) and a non-functional perspective (how well it does it).

Functional and non-functional requirements are verified differently. For example, FR-09 (Send Message) can be verified by checking whether a sent message reaches the recipient, while NFR-09 (Realtime Responsiveness) depends on the communication architecture and observed response time. This is why functional requirements are mainly verified through implemented features and tests, while non-functional requirements are also addressed through architectural and implementation decisions.

Requirements management also connects the project goals to the technical architecture described in Chapter 3. The requirement for real-time communication led to WebSockets; the requirement to support several categories of data led to the polyglot-persistence design (Oracle, MongoDB, Redis); the requirement to run the same product on any device led to the web client being the primary, universal client.

Project Requirements and Scope. The primary requirement of NexusTeam is to provide a centralized communication platform that registered users reach through either of two client applications. The system should let users create accounts, authenticate, find other users, create conversations, exchange messages, and manage their conversations, plus attachments, presence, chat organization, and voice communication.

The backend must provide one common API surface consumed by both the web client and the WPF desktop client. The system scope divides into five major areas:

1. User management (registration, authentication, sessions, profiles, search).
2. Communication and chat management (chats, groups, folders).
3. Messaging (sending, editing, deletion, delivery/read status, attachments).
4. Realtime infrastructure (WebSocket presence, typing, chat events, call signaling).
5. Supporting features (preferences, translation, AI image generation, themes).

End Users. End users are the primary stakeholders. They interact through the web client (the default, no-install option) or the WPF desktop client. Both support the core communication workflows, voice messages, and live calls, although the media implementations differ: browser WebRTC provides audio/video and screen sharing on the web, while the WPF client provides live audio using NAudio and WebSocket call communication. Their main expectations are simple interaction, reliable message delivery, fast response times, and secure authentication.

Development Team. The team consists of four members with distinct responsibilities: requirements/architecture/documentation (Sofiia Khyzhnychenko), backend and client implementation (Vladyslav Zaplitniy), user interface (Anna Kornet), and testing (Halil Hakan Karabay). This separation lets different technical areas move in parallel while sharing a common architecture (Chapter 3) and shared contracts (NexusTeam.Shared).

Project Supervisors / Evaluators. As an academic project, the system is evaluated by supervisors. Their requirements include a clear software architecture, appropriate documentation, evidence of testing, understandable project management, and a working, deployed implementation.

System Administrators / Developers. Developers and administrators need to install, configure, monitor, debug, and maintain the system — including the live production deployment on a VM. Installation, architecture, security, server, client, and deployment documentation are therefore all first-class project deliverables (Chapter 8).

Stakeholder needs can conflict. For example, users expect simple authentication, while administrators require device binding and rate limiting. Local development favors simple defaults, while production requires stricter secret validation. These differences were addressed by applying the appropriate requirements to each environment and by keeping the requirements traceable to architecture, implementation, and testing.

Functional Requirements

Functional requirements are identified with IDs so they can be referenced from later chapters and from test cases (Chapter 6).

FR-01 — User Registration. The system shall allow a new user to create an account by providing the required registration information. The server validates the request and creates the corresponding user record, rejecting invalid or duplicate accounts.

FR-02 — User Login. The system shall allow registered users to authenticate using their account credentials. After successful authentication the server issues a JWT access token and a refresh token. Authentication is implemented centrally so both clients use the same mechanism.

FR-03 — Session and Token Management. The system shall maintain authenticated sessions using access and refresh tokens, and shall support logout and token invalidation/revocation (refresh tokens are tracked in Redis so they can be revoked server-side).

FR-04 — User Profile Management. The system shall allow users to view and update their profile information and avatar. Profile data is accessed only through authorized operations tied to the authenticated user.

FR-05 — User Search. The system shall provide a way to find other users before starting a direct conversation.

FR-06 — Create Chat. The system shall allow users to create private, group, or channel-type conversations. The server validates the creation request (participant list, chat type) and stores the resulting chat.

FR-07 — Manage Chat. The system shall support retrieving chat information and managing participants and chat properties. Group deletion is restricted to the chat owner; direct-message chats have no separate "owner" concept. Only participants may read or modify a chat.

FR-08 — Chat Folders. The system shall allow users to organize conversations into folders, implemented as a dedicated backend component (ChatFolderService, MongoChatFolderRepository) and reflected in both clients.

FR-09 — Send Message. The system shall allow authenticated participants to send messages containing a sender, a target conversation, content (up to 4000 characters), and optionally a reply reference or attachments.

FR-10 — Real-Time Delivery. Messages shall be delivered to connected participants in real time over the WebSocket channel rather than through polling.

FR-11 — Edit Message. An authorized user (the sender) shall be able to modify a previously sent message; the edit timestamp is recorded and the change is broadcast to other connected clients.

FR-12 — Delete Message. The system shall support soft-deleting messages (an isDeleted flag, not a hard row delete), controlled by authorization rules and reflected to connected clients.

FR-13 — Message Status. The system shall track and broadcast message delivery and read status. FR-13a — Reactions. The system shall allow users to react to a message with an emoji, stored as a map of emoji to the list of reacting user IDs.

FR-14 — File Attachments. The system shall allow users to attach files (images, video, audio, documents, archives, code, up to 100 MB) to messages.

FR-15 — Attachment Retrieval. Authorized participants of a conversation shall be able to access its attachments, under the same authorization rules as messages.

FR-16 — WebSocket Communication. The client shall establish an authenticated WebSocket connection (/ws, JWT passed at connection time) for bidirectional realtime communication.

FR-17 — Typing Indicators. The system shall broadcast typing-started/typing-stopped events to other participants of a conversation.

FR-18 — User Presence. The system shall track and broadcast online/away/offline/DND status and last-seen time.

FR-19 — Chat Events. The realtime subsystem shall support events for users joining/leaving a conversation and other chat-state changes, so connected clients keep a synchronized view.

FR-20 — Call Initiation. The system shall allow a user to initiate a voice-call session with another participant via dedicated WebSocket call-signaling contracts (offer/answer/ICE-candidate exchange). **FR-21 — Call Control**. The system shall support the main call lifecycle operations: ringing, accept, reject, hang up, and busy/failure states. Call signaling is implemented server-side and shared by both clients. The web client uses browser WebRTC for audio/video media, ICE, and screen sharing, with coturn providing TURN relay support in production. The WPF client uses NAudio for live audio capture/playback over the WebSocket call channel. **FR-21a — Call History.** The system shall record the outcome of each call (completed, rejected, missed, failed) per chat, expose recent history to participants, and surface unseen missed calls to the callee on reconnect.

FR-22 — User Preferences. The system shall let users store and retrieve personal preferences (theme, language, notifications, privacy, chat display), implemented separately from core messaging in user_preferences.

FR-23 — Generated Images. The system shall let users generate images from a prompt inside a conversation, backed by a dedicated service, controller, and repository, and stored as attachments.

FR-24 — Translation Support.The WPF client shall be able to request translated message content through the dedicated DeepL-based TranslationService. The current web client does not expose this feature.

FR-25 — Voice Messages. Both clients shall allow users to record and send voice messages within a conversation; recipients shall be able to play the recorded audio. The web client records via the browser MediaRecorder API.

FR-26 — Local Chat Organization — see FR-08; folder organization is a server-persisted, per-user preference rather than a separate conversation entity.

FR-27 — Light and Dark Themes. Both clients shall allow switching between light, dark, and auto themes, persisted as a user preference and restored on next login.

Non-Functional Requirements

Non-functional requirements are grouped as: security, performance, reliability, usability, maintainability, scalability, compatibility, and deployment. Each category exists because a category of stakeholder cared about it directly: security and reliability trace back to end users trusting the platform with private conversations; performance and scalability trace back to the project supervisors' expectation that a "real-time" messenger actually behave like one under a classroom-sized load; maintainability and compatibility trace back to the development team itself, since four people working in parallel on shared code cannot afford an architecture that fights them; and deployment traces back to the

newest stakeholder concern this document describes — a system administrator who has to keep the thing running on a VM after the team has moved on to the next milestone.

Security Requirements.

NFR-01 — Secure Password Storage. Passwords are never stored as plaintext; BCrypt hashing (work factor 11) is used.

NFR-02 — Token-Based Authentication. All authenticated API and WebSocket operations are protected with JWT bearer tokens.

NFR-02a — Device Binding (Device Lock). The system shall optionally bind an authenticated session to a specific device, protected by a PIN (hashed with a server-side pepper). Once enabled, requests outside /api/auth, /api/device-lock, and /health from an unrecognized or locked device are rejected with 401, even if the JWT itself is valid.

NFR-03 — Resource-Level Authorization. Every chat, message, attachment, and generated-image operation is checked against chat membership or resource ownership before it executes; anonymous callers are rejected with 401 rather than silently allowed (IResourceAuthorizationService, hardened per the project TODO.md).

NFR-04 — Rate Limiting. Login attempts and message sends are rate-limited per user/identifier to slow brute-force and abuse.

NFR-05 — Transport Security. Production traffic is served over HTTPS/WSS via a trusted Let’s Encrypt certificate; internal API and database ports are bound to localhost and not exposed publicly.

Performance Requirements.

NFR-09 — Realtime Responsiveness. Messages and other realtime events shall be delivered with minimal delay over an active WebSocket connection — the main reason WebSocket is used instead of polling.

NFR-10 — Efficient Data Access. The system shall use appropriate database technologies and indexes; MongoDB provides indexed document access for communication data, Redis provides sub-millisecond access for temporary/high-frequency data.

Reliability Requirements.

NFR-12 — Connection Recovery. Clients shall recover from temporary WebSocket disconnects via an automatic reconnection strategy.

NFR-13 — Offline Message Handling. The WPF client shall queue outgoing messages while disconnected and flush the queue on reconnect through OfflineMessageQueue. The web client reconnects automatically and reloads server-held history but does not maintain the same local offline queue.

NFR-14 — Health Checks. The server shall expose liveness/readiness health-check endpoints so the deployment pipeline and infrastructure can detect a broken release before routing traffic to it.

Usability Requirements.

NFR-16 — Intuitive Navigation. Users shall be able to move between conversations, settings, and profile without ambiguity.

NFR-17 — Consistent User Experience. The main communication concepts shall stay consistent between the web and WPF clients even though their technologies and layouts differ, because both consume the same backend.

NFR-18 — Feedback. The system shall give users visible feedback for in-progress and failed operations (loading states, error messages, delivery/read ticks).

Maintainability Requirements.

NFR-19 — Separation of Responsibilities. Presentation, business logic, and data access are kept in separate layers (controllers/services/repositories on the backend, views/view models/services on the WPF client).

NFR-20 — Shared Contracts. Common models, DTOs, enums, and WebSocket contracts are centralized in NexusTeam.Shared to avoid duplication and drift between server and clients.

Scalability Requirements.

NFR-22 — Database Scalability. Communication data uses a document model suitable for large datasets; the MongoDB deployment is a sharded cluster (config server, two shards, mongos router) rather than a single instance.

NFR-23 — Infrastructure Scalability. The system shall be deployable as independently scalable containerized components (Docker Compose in development and production).

Compatibility Requirements.

NFR-25 — Desktop Compatibility. The WPF client targets Windows with the .NET 8 runtime.

NFR-26 — Web Compatibility. The web client shall run in any modern desktop or mobile browser without installation, and shall be responsive (side-by-side layout on desktop, full-screen navigation on screens ≤768px).

NFR-27 — Common Backend. Both client types use the same backend services rather than duplicating business logic per platform.

Deployment Requirements.

NFR-28 — Containerized Deployment. The full stack (server, web client, Oracle, MongoDB cluster, Redis, seeder) shall be deployable with Docker Compose, both for local development and for production.

NFR-29 — Database Initialization. NexusTeam.DbSeeder shall initialize database schemas/collections and development data on first startup.

NFR-30 — Production Deployment. The system shall be deployable to a remote Linux virtual machine over SSH/VPN using a repeatable, scripted process (deploy.sh) that builds new images, performs a health-checked cutover, provisions a trusted TLS certificate, and can roll back automatically if the new release fails its health checks, without destroying the persistent Oracle/MongoDB/Redis volumes.

Requirements Rationale and Edge Cases. Some requirements reflect specific implementation decisions. **FR-12 (Delete Message)** specifies a soft delete rather than a hard row delete deliberately: a messaging platform that hard-deletes rows loses the ability to show "this message was deleted" to the other participant, loses any audit trail for moderation, and complicates the realtime broadcast (a client that receives a delete event for a message it never actually loaded needs something to reconcile against). **FR-07 (Manage Chat)** restricting group deletion to the chat owner, while leaving direct-message chats without an equivalent "owner" concept, reflects a real asymmetry in the domain rather than an oversight — a DM has exactly two symmetric participants and no natural single owner, while a group chat has a creator whose removal shouldn't silently orphan the group's administrative capabilities. FR-16 through **FR-19 (the realtime requirements)** are deliberately scoped to events, not to guarantees — the requirements describe what gets broadcast, not a guarantee that every client receives every event exactly once, because that stronger guarantee would require a different architecture (message acknowledgment, at-least-once delivery with client-side deduplication) that the project judged out of scope for its timeline; NFR-12 and **NFR-13 (connection recovery and offline queuing)** are the compromise that keeps this scoping honest, since they at least guarantee that a disconnected client eventually catches up rather than silently losing messages forever.

NFR-04 (rate limiting) and **NFR-02a (device lock)** are independent mechanisms and can both affect a user during authentication. A user who enables device lock and then reaches the login rate limit while re-authenticating from a new device can therefore encounter both restrictions. The mechanisms are kept independent so that each can be configured and tested separately; the resulting interaction is documented as a limitation of the current user flow.

Requirements Process, Traceability, and Risk

- **High priority (core system):** user registration, authentication, chat creation, sending/receiving messages, real-time delivery, secure password storage, containerized deployment.
- **Medium priority:** message editing/deletion, attachments, presence, typing indicators, chat folders, user preferences, rate limiting, production deployment to a VM.
- Lower priority / extended scope:translation, AI image generation, voice messages, live audio/video calls, screen sharing, themes, reactions, message search, forwarding, and advanced chat-management features.

Requirements Traceability. Requirements traceability connects each requirement to the component that implements it and to the tests that verify it, for example:

- **FR-10 (Real-Time Delivery)** → Architecture: WebSocket layer, §3.14–3.16 → Testing: §6.13.
- **FR-20/21 (Voice Calls)** → Architecture: call signaling + coturn, §3.14, §3.28 → Testing: §6.19.
- **NFR-30 (Production Deployment)** → Architecture: §3.28, Figure 13 (Production Deployment Architecture) → Documentation: §8.17.

- **FR-08 (Chat Folders)** → Architecture: ChatFolderService/MongoChatFolderRepository, §3.9 → Implementation: §4 → Testing: §6.18.
- **FR-14/15 (Attachments)** → Architecture: AttachmentService, server-side file storage, §3.9 → Implementation: §4.13 → Testing: §6.17.
- **NFR-02a (Device Lock)** → Architecture: DeviceLockMiddleware, §3.23, §3.29 → Implementation: §4.26a → Testing: §6.24.
- **NFR-22 (Database Scalability)** → Architecture: sharded MongoDB cluster, §3.11 → Documentation: §8.11.

The traceability table links requirements to the related architecture, implementation, testing, or documentation. The same approach was used when requirements such as device lock and call history were added during development.

Requirements and Architectural Decisions. Several architectural decisions trace directly to these requirements. Real-time messaging led to WebSockets over polling. Supporting several categories of data led to Oracle + MongoDB + Redis. Supporting two very different clients on one backend led to a centralized server with shared contracts. Security requirements shaped JWT/refresh-token authentication, BCrypt hashing, rate limiting, security headers, resource-level authorization, and WebSocket authentication. The new production-deployment requirement led to the addition of a TLS gateway, Let's Encrypt automation, and a scripted, health-checked deployment procedure — described in Chapter 3.

Requirements Management Process. Requirements were managed continuously rather than only at project start, through four stages: identification of required functionality and quality characteristics; analysis of dependencies, technical constraints, and feasibility; prioritization into the levels in §2.20; and assignment of requirements to project components and team responsibilities.

Requirements Constraints.

- **Technical constraints.** The project integrates ASP.NET Core, two clients (web + WPF), WebSocket, Oracle, a sharded MongoDB cluster, Redis, and Docker — each with its own operational learning curve.
- **Time constraints**. As an academic project, development happened within a fixed period, which required prioritizing core functionality over optional extensions such as end-to-end encryption.
- **Team constraints.** With four team members covering requirements/architecture, backend/client implementation, UI, and testing, work had to be planned so each area could progress independently against the shared contracts in NexusTeam.Shared.
- **Infrastructure constraints.** Running three separate database technologies simultaneously (Oracle, a sharded MongoDB cluster with three separate mongod processes plus a router, and Redis) is heavier than a typical student project's development environment, which shaped several downstream decisions: NexusTeam.DbSeeder exists specifically to make that heavy environment reproducible with a single command rather than a manual setup guide per database (§4.41), and the 8 GB RAM recommendation in §1 reflects the resource requirements of this multi-database design.
- **Knowledge constraints.** No single team member arrived already expert in all of ASP.NET Core, WPF/MVVM, MongoDB sharding, Oracle administration, Docker Compose networking, and WebRTC signaling — each of these was, to some degree, learned during the project rather than known in advance. This shaped which requirements were treated as core versus extended: the features requiring the least unfamiliar technology (chat CRUD, REST messaging) were implemented first, both because they were higher priority (§2.20) and because they gave the team a working system to build unfamiliar pieces (WebSocket

realtime, then voice signaling) on top of, incrementally, rather than attempting everything simultaneously.

Requirements Risks.

- **Scope expansion.** A messaging system can grow indefinitely (video calls, reactions, encryption, notifications, integrations). The project mitigated this by fixing a defined set of core and extended requirements and tracking further ideas as future work (Chapter 9) rather than mid-project scope creep.
- **Technical complexity.** Multiple databases and communication technologies increase operational complexity; this was accepted as a deliberate trade-off (§3.36).
- **Deployment risk.** Shipping to a real VM introduces exposure the local Docker setup does not have (public network, certificates, secrets). This was mitigated by restricting access through a VPN, binding internal ports to localhost, and scripting the deployment with automatic rollback on failed health checks.
- **Requirement/implementation drift.** A requirement can be satisfied when written but silently stop being satisfied as the implementation evolves around it — for example, a resource-authorization gap that opens up as new endpoints are added without the same authorization check being applied consistently. This is exactly the class of risk the TODO.md hardening milestones referenced throughout this document (§3.29, §7.15) were closing, and it is also the reason requirements traceability (above) is treated as a living check rather than a one-time exercise performed only when the requirement was first written.
- **Documentation risk.** Documentation can become outdated even when the implementation and tests are correct. A rotating documentation-owner role was introduced to reduce this risk and keep architecture, testing, and deployment information aligned with the implementation.

Summary. Requirements management established the functional and quality objectives of NexusTeam and grounded the architecture and implementation decisions that follow. The most important functional requirements are secure authentication, chat management, realtime messaging, message management, attachment handling, user presence, and voice communication (voice messages on both clients, live audio/video calls and screen sharing on the web client, and live audio calls on the WPF client). Additional functionality — chat folders, preferences, translation, and image generation — extends the core communication capabilities. Non-functional requirements focus on security, realtime performance, reliability, maintainability, usability, scalability, compatibility, and — now that the system runs on a production VM — deployability. Traceability connects requirements through architecture, implementation, and testing. The next chapter describes how these requirements were translated into the overall system architecture.

3. Architecture

Architectural Overview and Goals

The architecture of NexusTeam supports the requirements described in Chapter 2 while keeping a clear separation of responsibilities between the different parts of the system.

NexusTeam follows a client-server architecture in which the backend is the single point of communication between clients and the persistence/infrastructure layer. Two client applications are provided: a responsive web client built with HTML, CSS, and JavaScript and served by Nginx (the primary, universal client), and an optional WPF desktop client for Windows, used mainly as an optional native Windows client.

The backend is implemented with ASP.NET Core 8 and exposes both REST HTTP endpoints and a WebSocket-based realtime channel. REST is used for conventional request-response operations; WebSocket provides persistent bidirectional communication for realtime events and call signaling.

The system uses three infrastructure technologies: Oracle Database for structured user data, a sharded MongoDB cluster for document-oriented communication data, and Redis for caching, sessions, presence, and rate limiting.

The overall architecture is a combination of:

- client-server architecture, with two interchangeable clients on one backend;
- a layered backend architecture (controllers → services → repositories);
- polyglot persistence (Oracle + MongoDB + Redis);
- a component-based front-end architecture for the web client, and MVVM for the WPF client;
- containerized deployment, including a production topology deployed to a virtual machine.

Architectural Goals.

Separation of Responsibilities. UI code never accesses databases directly; database-specific code never mixes with presentation logic. Controllers, services, repositories, and infrastructure components each have one job.

Reusability. The same backend serves both clients. Shared DTOs and WebSocket contracts in NexusTeam.Shared remove duplication between server and clients.

Maintainability. Components can be modified independently: controllers, services, repositories, clients, and infrastructure have distinct, narrow responsibilities.

Real-Time Communication. The architecture is built around a persistent WebSocket channel rather than around polling, so messaging, presence, typing, and call signaling can all be pushed to clients immediately.

Operability. The system must run the same way, container-for-container, on a developer laptop and on the production VM, with a scripted, health-checked path from a committed Git revision to a running deployment.

High-Level System Architecture. At the highest level the system consists of client applications, an application server, and infrastructure components .

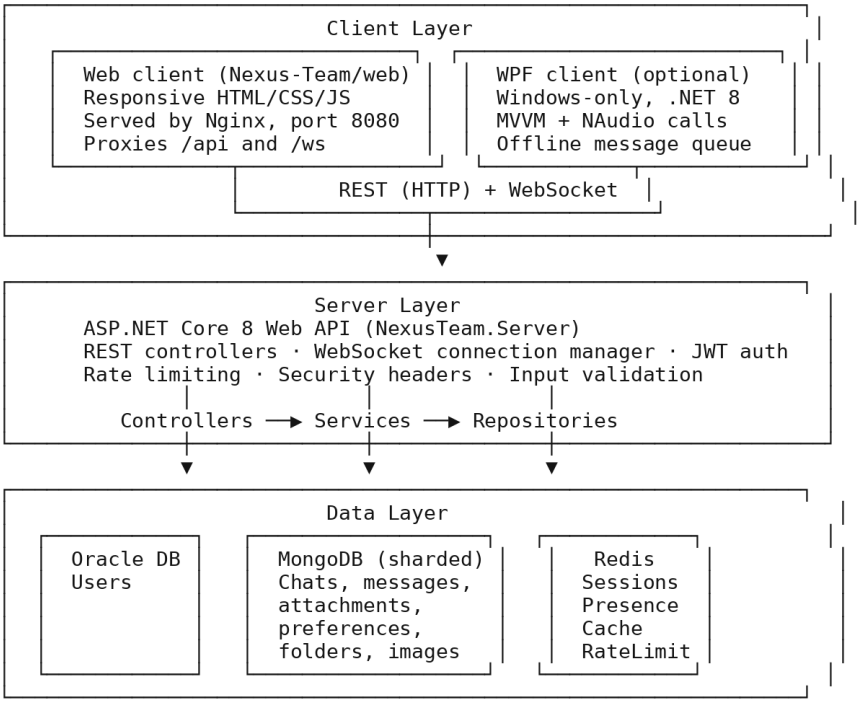


Figure 2 — High-Level System Architecture.

Client-Server Architecture. NexusTeam uses a centralized backend that provides services to multiple clients. The clients are responsible for:

- presentation and user interaction;
- local UI state (open chat, draft message, theme selection);
- client-side services (WebSocket client, offline queue, credential storage);
- all communication with the server (no direct database access).

The backend is responsible for:

- authenticating and authorizing every request;
- implementing all business logic (chat/message rules, folder management, preferences, image generation, call signaling);
- mediating all access to Oracle, MongoDB, and Redis;
- broadcasting realtime events to every relevant connected client.

Backend Layered Architecture

The backend is an ASP.NET Core 8 application organized into logical layers: Controllers (HTTP boundary) → Services (business logic) → Repositories (data access), plus a cross-cutting Middleware pipeline and a WebSocket subsystem that sits alongside the REST controllers.

Controllers Layer. Controllers represent the HTTP API boundary. The server has ten controllers covering the major functional areas:

- AuthController — registration, login, logout, token refresh.
- UsersController — user profile management, avatar upload, user search.
- ChatsController — chat creation, management, participant management.
- MessagesController — message search; message creation, editing, deletion, and history are exposed from ChatsController under the chat resource routes.
- AttachmentsController — file upload, download, thumbnail retrieval, message attachment lookup, update, and deletion.
- PreferencesController — reading and updating user preferences.
- ChatFoldersController — chat folder organization.
- GeneratedImagesController — AI-generated image requests and retrieval.
- CallsController — call-history recording/retrieval, unseen-missed-call notifications, and ICE-server discovery.

Service Layer. The service layer holds the main business logic.

Core (domain) services:

- **AuthService** — authentication, registration, password management.
- **ChatService** — chat management, participants, ownership rules, pinning, and chat metadata.

- **ChatService** — chat management and participants.
- **MessageService** — message handling and history.
- **ChatFolderService** — chat folder organization.
- **AttachmentService** — file attachment handling.
- **AvatarService** — avatar image processing.
- **GeneratedImageService** — AI image generation management.
- **CallHistoryService** — recording and retrieving call outcomes (§4.26b).

Infrastructure services:

- **WebSocketConnectionManager** — realtime connection registry, §3.16.
- **UserStatusService** — user presence tracking.
- **SessionService** — session lifecycle management.
- **RateLimitService** — API rate limiting (§4.25).
- **RedisCacheService** — general-purpose caching operations.
- **PresenceTrackingService** — background service reconciling presence state (§4.21).
- **UserDeviceService** — device registration and lock state for device binding (§4.26a).

Repository Layer. The repository layer abstracts database access from business logic.

- **Oracle** — one repository: OracleUserRepository, user data persistence.
- **MongoDB** — eight repositories: MongoChatRepository (chats), MongoMessageRepository (messages), MongoMessageAttachmentRepository (attachment metadata), MongoUserPreferenceRepository (preferences), MongoChatFolderRepository (folders), MongoGeneratedImageRepository (generated images), MongoCallHistoryRepository (call history), and MongoUserDeviceRepository (device state).

Persistence Architecture

One of the central architectural decisions in NexusTeam is using multiple storage technologies instead of one database for everything. The three infrastructure components reflect different characteristics of the stored data: structured, low-volume account data (Oracle); flexible, high-volume communication documents (MongoDB); and fast-access, short-lived state (Redis).

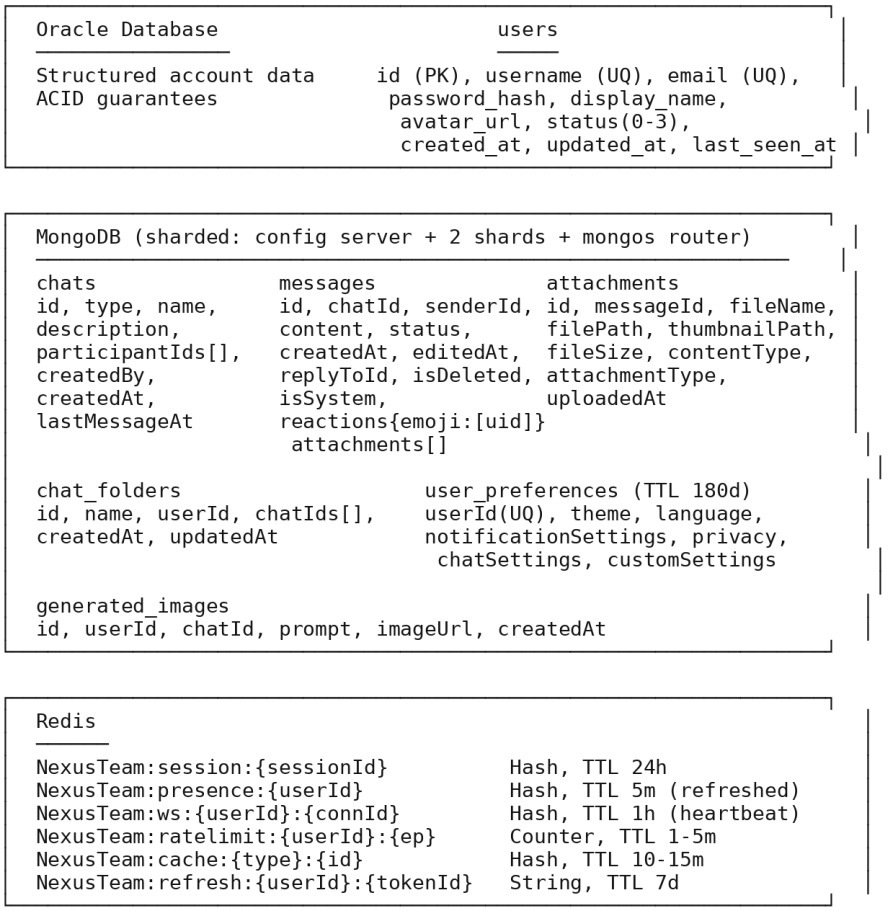


Figure 3 — Database Diagram (Polyglot Persistence).

Oracle Database. Oracle is used for structured user-related information. The server has an OracleDataContext (accessed through IOracleDataContext) that manages Oracle connections via the Oracle Managed Data Access provider. The single users table

enforces uniqueness on username/email and a check constraint on status (0=Offline, 1=Online, 2=Away, 3=DND); the access flow runs from the calling service down through the repository to the database.

Figure 4 — Oracle Access Flow.

MongoDB. MongoDB is used for document-oriented communication data: chats, messages, message attachments (metadata — the files themselves live on the server file system), chat folders, user preferences, generated images, call history, and user-device state. Production runs MongoDB as a sharded cluster (config server, two shards, mongos router) rather than a single instance; development uses the same topology in Docker for parity.

Redis. Redis is a separate infrastructure layer for data that benefits from fast access and does not need durable, primary storage. The server registers IRedisMultiplexer, ICacheService, and IUserStatusService. Redis backs sessions, refresh tokens, presence, general caching, and rate-limit counters (see Figure 3).

Realtime and API Communication Architecture

REST is used for conventional request-response operations: registration, login/logout/refresh, user profile/status and avatar operations, chat retrieval and management, message history/search, attachments, preferences, folders, generated images, call history, device-lock management, and ICE-server discovery. See §4 for the endpoint list.

WebSocket Architecture. REST alone is not sufficient for a realtime messenger, because the client would otherwise have to repeatedly poll for updates. NexusTeam

therefore implements a dedicated WebSocket layer: the server enables WebSockets and routes connections to the /ws endpoint.

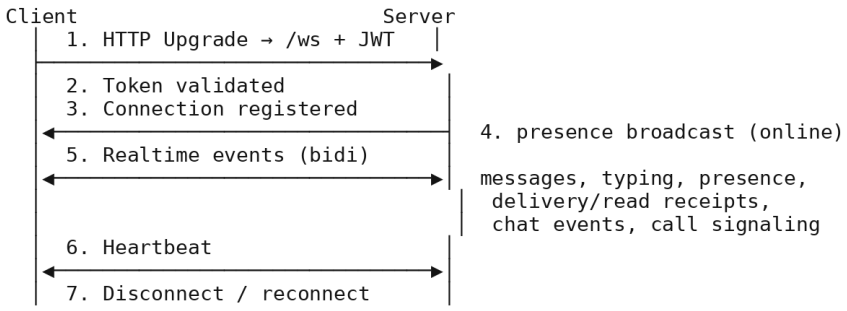


Figure 5 — WebSocket Connection and Message Flow.

WebSocket Message Envelope. Realtime communication is standardized with a shared WebSocket message envelope. The shared project defines WebSocketMessageEnvelope and WebSocketMessageType plus contracts for chat messages, delivery/read status, typing, presence, errors, and calls. The call set includes CallRequestContract, CallAnswerContract, CallRejectContract, CallEndContract, CallSdpOfferContract, CallSdpAnswerContract, CallIceCandidateContract, CallAudioDataContract, and CallTimeoutContract. Edit, delete, and reaction operations use the corresponding WebSocketMessageType values and payload DTOs.

```
WebSocketMessageEnvelope { type: WebSocketMessageType, payload: <typed contract> }
```

Figure 6 — WebSocket Message Envelope Structure.

WebSocket Connection Management. Active connections are managed by a dedicated WebSocketConnectionManager, implementing IWebSocketConnectionManager. It abstracts currently connected users and their connections — a user may have zero, one, or several simultaneous connections (e.g. web + WPF at once). Connection registration

is atomic: a connection ID cannot be attached to two different users. Application services send messages to a user through the manager without touching raw WebSocket objects.

```
Application Service → IWebSocketConnectionManager → WebSocket(s) for user
```

Figure 7 — WebSocket Connection Management Flow.

Shared Architecture. NexusTeam.Shared provides definitions that must stay consistent between server and clients:

- **Domain models**— User (account and status), Chat (direct/group/channel, participants, metadata), Message (content, status, reply/forward snapshots, deletion/system flags, reactions, attachments), MessageAttachment (file metadata), and ChatFolder (per-user chat grouping). Additional persistence models cover call history and user-device state.
- **DTOs** — request/response shapes for every REST endpoint (§4.42).
- **WebSocket contracts** — the typed realtime envelope payloads (§3.15).
- **Enums** — UserStatus, ChatType, MessageStatus, AttachmentType, WebSocketMessageType.
- **Custom exceptions** — UserNotFoundException, ChatNotFoundException, and similar domain exceptions the middleware maps to HTTP status codes (§4.39).

Client Architectures

The primary NexusTeam client is browser-based, implemented with plain HTML for structure, CSS for presentation and responsive layout, and JavaScript for application logic, state management, communication, authentication, messaging, attachments, voice-message recording, folders, themes, notifications, image generation, live audio/video

calls, ICE/TURN configuration, and screen sharing. No SPA framework such as Angular or React is used.

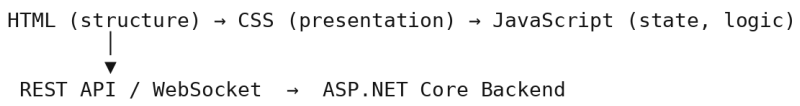


Figure 8 — Web Client Request Flow.

This keeps the browser presentation layer separate from backend business logic and persistence. Nginx serves the static files and proxies /api and /ws to the backend, so the client is delivered as a single container reachable on port 8080.

The web client does not use a framework such as Angular or React. The client is relatively small and communicates with one backend, so the project uses plain HTML, CSS, and JavaScript without adding a separate build and framework layer. State management, DOM updates, and WebSocket handling are implemented in app.js. This keeps the deployment simple, although it also means that the client has to maintain these structures directly as the application grows.

WPF Desktop Client Architecture. The optional WPF client follows the MVVM pattern. The repository currently contains 21 XAML views/windows/controls and 26 ViewModel classes, including supporting item and selector ViewModels. CommunityToolkit.Mvvm provides the MVVM infrastructure. Client services wrap backend integration, including AuthenticationService, MessagingService, CallService, TranslationService, and file/avatar services. The WPF call implementation uses WebSocket call messages and NAudio rather than a browser-style WebRTC media stack.

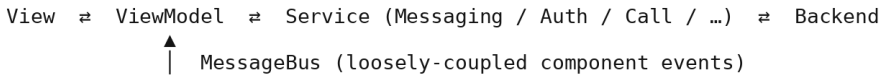


Figure 9 — WPF MVVM Architecture.

Offline and Reconnection Architecture. Realtime communication depends on network connectivity, so connection failures must be handled. Both clients implement automatic WebSocket reconnection. The WPF client additionally buffers outgoing messages in an OfflineMessageQueue while disconnected and flushes them once the connection is restored.

Voice Call Architecture. Voice messages are recorded client-side (browser MediaRecorder on the web client; NAudio capture on the WPF client) and sent as ordinary attachments. Live web calls use WebSocket call signaling to exchange call-control, SDP and ICE information, then establish browser WebRTC audio/video media; the web UI also supports screen sharing. The WPF client uses WebSocket call messages and NAudio for live audio capture/playback. The production coturn relay provides TURN connectivity for browser WebRTC when a direct peer-to-peer path is unavailable.

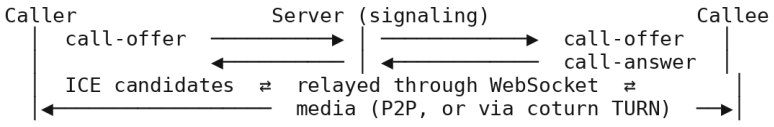


Figure 10 — Voice Call Signaling Flow.

Authentication Architecture. Authentication is centralized on the backend. The main components are AuthService, JwtTokenService, RefreshTokenService, SessionService, BcryptPasswordHasher, and JwtAuthenticationMiddleware.

Cross-Cutting Concerns: Middleware, Validation, Dependency Injection, and Configuration

The backend uses a custom middleware pipeline configured for security headers, exception handling, request logging, JWT authentication, device-lock enforcement, and WebSocket handling.

```
Request → SecurityHeaders → ExceptionHandling → RequestLogging
        → JwtAuthentication → DeviceLock (if enabled) → WebSocketHandler (if /ws) → Controller
```

Figure 11 — Middleware Pipeline Flow.

DeviceLockMiddleware lets exactly the routes it must — /api/auth, /api/device-lock, and /health — bypass the check, and otherwise requires the request to carry a device ID that is currently unlocked for that user (§3.29, §4.26a).

Each middleware component in the pipeline has one job:

- **SecurityHeadersMiddleware** — injects X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, and CSP headers on every response.
- **ExceptionHandlingMiddleware** — converts unhandled/domain exceptions into the correct HTTP status without leaking internal details (§4.39).
- **RequestLoggingMiddleware** — logs request/response metadata via Serilog (§4.40).
- **JwtAuthenticationMiddleware** — validates the bearer token and populates HttpContext.Items["UserId"].
- **DeviceLockMiddleware** — enforces device binding, as above (only active where device lock is enabled for the user).

- **WebSocketHandler** — upgrades and manages /ws connections (messaging and call signaling), §3.14, §4.17.

Validation Architecture. Input validation is a separate concern: FluentValidation validators are registered and automatically applied to incoming requests, so invalid requests are rejected before reaching business logic.

```
Request → FluentValidation → (invalid → 400) → Controller → Service
```

Figure 12 — Validation Flow.

Dependency Injection. Dependency injection is used throughout the backend. Interfaces such as IUserRepository, IChatRepository, IMessageRepository, IAuthService, IMessageService, and IWebSocketConnectionManager are registered in the DI container rather than being constructed directly by their consumers, which keeps components loosely coupled and testable.

Docker and Deployment Architecture. Docker provides a reproducible infrastructure environment for the server, web client, Oracle, MongoDB (config server + shards + router), Redis, and the one-shot database seeder. Production additionally runs an Nginx/Certbot TLS gateway and a coturn TURN relay. See §3.28 for the production topology and Chapter 8 for the deployment procedure itself.

Configuration Architecture. Configuration is separated from application logic through dedicated option classes for Oracle, MongoDB, Redis, JWT, BCrypt, rate limiting, CORS, and device lock, bound from appsettings.json in development and from environment variables/.env.production in production. Every option class has a matching validator (OracleOptionsValidator, MongoOptionsValidator, RedisOptionsValidator, JwtOptionsValidator, BcryptOptionsValidator, RateLimitOptionsValidator, CorsOptionsValidator, DeviceLockOptionsValidator) that runs at application startup, so

a misconfigured connection string or an accidentally-empty JWT secret fails fast with a clear error instead of surfacing later as a confusing runtime exception. The overall shape of appsettings.json mirrors the option classes directly — a ServerConfiguration block for the listening port, an Oracle block (connection string, command timeout, retry attempts), a MongoDB block (connection string, database name, connection and server-selection timeouts), a Redis block (connection string, database index, connect/sync timeouts, AbortOnConnectFail), a Jwt block (secret key, issuer, audience, access-token and refresh-token lifetimes), a Bcrypt block (work factor), a RateLimit block (login and message attempt/window pairs), and a Cors block (allowed origins, credentials, policy name) — so a developer reading the configuration file already knows, from its structure alone, which part of the architecture each setting belongs to.

Deployment and Security Architecture

The system is deployed to a Linux virtual machine reached over a VPN, using deploy.sh (see §8.17 for the full procedure), with only the TLS gateway and TURN relay exposed publicly and everything else bound to localhost.

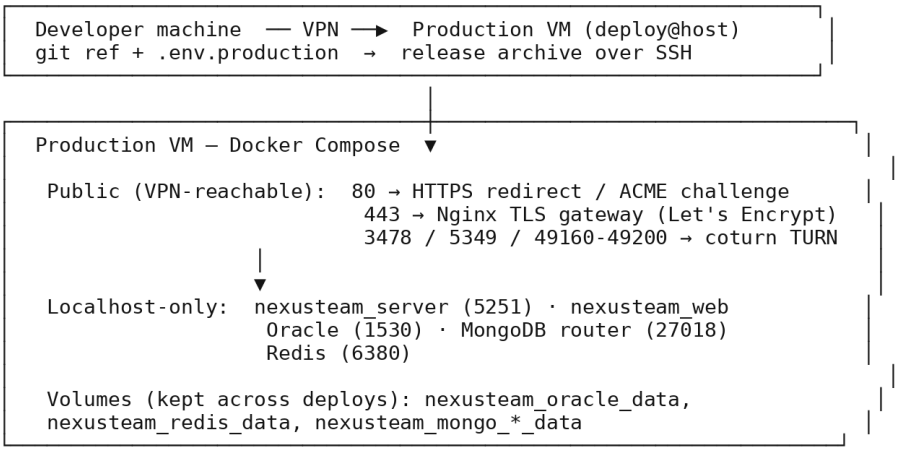


Figure 13 — Production Deployment Architecture.

Only the TLS gateway and the TURN relay ports are public; the API, web container, and every database are bound to 127.0.0.1 and reachable only from inside the VM or

over the VPN. deploy.sh builds new images alongside the running stack, cuts over once nexusteam_server and nexusteam_web report healthy, and rolls the previous release back automatically if they don't.

Security Architecture. Security is implemented across multiple layers rather than in one place:

- **Authentication** — JWT access tokens plus Redis-backed refresh tokens establish authenticated sessions.
- **Password protection** — BCrypt hashing (work factor 11).
- **Authorization** — every chat/message/attachment/generated-image operation is checked against chat membership or ownership (IResourceAuthorizationService); anonymous access to previously open endpoints was closed off in a recent hardening pass (see the project TODO.md).
- **Device binding** — an optional second factor that ties a session to a specific device via a PIN, enforced by DeviceLockMiddleware/DeviceLockController, independent of the JWT itself (§2.12).
- **Rate limiting** — login and message-send endpoints are throttled per identifier via Redis counters.
- **Transport security** — production traffic is served over HTTPS/WSS with a Let's Encrypt certificate obtained and renewed automatically by Certbot; only the TLS gateway and the TURN relay are internet-facing.
- **Security headers** — X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, and a per-environment Content-Security-Policy; the Kestrel Server header is suppressed to avoid disclosing implementation details.

WebSocket connections get the same scrutiny as REST: the JWT is validated before the connection is accepted, the resulting user context is used to check chat participation before a message is accepted or relayed, and production traffic runs over WSS. Voice-

call signaling inherits this authentication — every offer, answer, and ICE candidate travels over the same authenticated channel, call requests are only forwarded to the intended recipient, and no call media is stored server-side (only the call-history metadata described in §4.26b). The current security posture has known, explicitly documented limits: there is no end-to-end encryption for messages, no two-factor authentication, and authorization is participant/ownership-based rather than a full role-based access control system — all three are tracked as future work (Chapter 9) rather than silently omitted.

At the operational level, the security model distinguishes what developers, administrators, and the deployment process are each responsible for. Developers are expected to never log sensitive data, validate all input through FluentValidation, rely on parameterized queries and BCrypt rather than ad hoc protection, and keep dependencies patched. Administrators are responsible for rotating default secrets (JWT signing key, database passwords), enabling database authentication in production, restricting Docker network access, and reviewing logs for security events. The deployment process itself keeps secrets in environment variables and the git-ignored .env.production file rather than in source control, isolates the Docker network, and — as described in §8.17 — refuses to proceed if it detects default or development-grade secrets. On the compliance side, the system supports the GDPR-relevant operations a messaging product needs: a user can retrieve their own data, delete their account (soft-deleted messages remain recoverable for a bounded period rather than vanishing instantly, which is itself a documented trade-off), and export their data; a lightweight incident-response procedure (identify → contain → eradicate → recover → learn) and a private vulnerability-reporting channel (no public issue for an unpatched security bug) round out the operational security posture.

Scalability Considerations. The current single-VM deployment (§3.28) is sized for a course project’s evaluation load rather than for production traffic at scale, but the architecture was deliberately kept compatible with horizontal growth rather than

designed only for the size it currently runs at. On the API side, the server is stateless by construction: JWT authentication means any server instance can validate a request without consulting shared session state, so a future move to multiple server instances behind a load balancer would not require re-architecting authentication — it would require adding the load balancer and, for WebSocket traffic specifically, either sticky sessions per connection or a Redis Pub/Sub layer so that a message delivered on one server instance can still reach a recipient whose WebSocket happens to be held open by a different instance. The database layer already has scaling paths built into the current design rather than deferred to a rewrite: MongoDB runs as a sharded cluster today, not as a future upgrade (§3.11); Oracle could add read replicas for read-heavy user lookups without touching the write path; and Redis could move to a Redis Cluster for high availability without changing how the application talks to it, since the application already accesses Redis through IRedisMultiplexer rather than a hardcoded single-instance client.

Several performance techniques are already in place rather than purely theoretical: message history uses cursor-based pagination capped at fifty messages per page rather than returning an unbounded result set; user profiles and chat details are loaded lazily, on demand, rather than eagerly joined into every response that merely references a chat or a user by ID; the WebSocket layer supports batching status updates (for example, marking several messages as read in one event) rather than emitting one event per message; and HTTP responses use gzip compression. Database access follows conventional but consequential discipline — indexes on the columns and fields that are actually queried (§3.10, §3.11), composite indexes for the query patterns that combine several of those columns (chat ID plus timestamp, for instance), connection pooling on both the Oracle and HTTP-client sides, and a cache-aside pattern in Redis with TTL-based expiration and explicit invalidation on writes, rather than letting stale cached data linger past its useful life. None of this makes claims about a specific supported user

count — that would require the load testing flagged as future work in §9.4.4 — but it does mean that scaling up, when it becomes necessary, is a matter of adding infrastructure within the existing architecture rather than redesigning the architecture itself.

Architectural Decisions, Strengths, and Evaluation

Using both REST and WebSocket, instead of one protocol for everything, was a deliberate choice. REST suits operations the client explicitly triggers — login, retrieving chat/message history, updating settings, uploading files. WebSocket suits everything the server needs to push without being asked — new messages, edits, deletions, typing, presence, delivery/read receipts, and call signaling.

Architectural Decision: Multiple Databases. Using Oracle, MongoDB, and Redis together increases infrastructure complexity compared with a single database, but it gives a clean separation between categories of data.

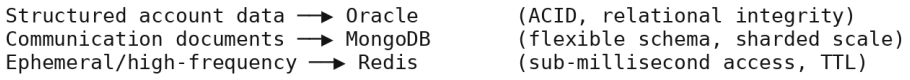


Figure 14 — Data Categories by Store.

Architectural Decision: Shared Project. NexusTeam.Shared avoids duplicated definitions between server and clients. Without it, the server and each client would independently define DTOs and realtime contracts, risking incompatibility.

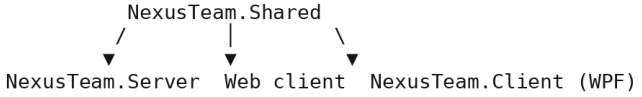


Figure 15 — Shared Project Dependency Diagram.

Architectural Decision: Two Clients, One Backend. Serving one universal web client and one optional native client from the same backend was chosen over building two independent backends or one framework that tries to target both surfaces at once. It keeps business logic in exactly one place, at the cost of the web client needing its own hand-rolled state management (no framework) and the WPF client needing its own MVVM plumbing.

Architectural Strengths.

- **Clear separation of concerns** — controllers, services, repositories, clients, and infrastructure each have distinct responsibilities.
- Two-client support from one backend, avoiding duplicated business logic.
- Realtime communication via WebSocket, appropriate for a modern messenger.
- **Database specialization** — each store is used for what it's good at.
- A real, scripted production deployment with health-checked cutover and rollback, not just a local Docker Compose file.

Architectural Limitations. Multiple databases increase operational complexity — developers must understand Oracle, MongoDB, and Redis, and deployment needs several infrastructure components. The WebSocket subsystem adds complexity beyond a purely REST-based app: connection lifecycle, authentication, reconnection, routing, and state all need management. Two clients increase the integration surface — a change to a shared DTO or WebSocket contract can affect the server and both clients simultaneously. The two clients also use different live-call media implementations: the web client uses browser WebRTC, while the WPF client uses NAudio over WebSocket. Finally, the demo-account seeder recreates fixed demo credentials on every startup, which is fine for a VPN-only demo but must be disabled or changed before any

exposure to untrusted users. These trade-offs were accepted because they support the project's functional and architectural goals.

Architecture-to-Requirements Mapping.

Requirement	Architectural response
FR-10 Real-Time Delivery	WebSocket layer and shared message contracts, §3.14–3.16
FR-20/21 Voice Calls	WebSocket call-control/signaling; WebRTC + coturn for web media; NAudio/WebSocket audio for WPF, §3.21
NFR-03 Resource Authorization	*IResourceAuthorizationService*, §3.29
NFR-22 Database Scalability	Sharded MongoDB, §3.11
NFR-30 Production Deployment	VM deployment architecture, §3.28

The mapping connects the main requirements with the architectural components used to implement them.

Summary. NexusTeam implements a modular client-server architecture built around realtime communication, two clients on one backend, data specialization, and maintainability. The ASP.NET Core 8 backend is the central application layer: controllers expose HTTP endpoints, services implement business logic, repositories abstract data access, and middleware handles authentication, logging, security, exceptions, and WebSocket communication.

The web client is the primary, browser-based, framework-free front end; the WPF client is an optional Windows desktop client with native UI, live audio calls, translation, and offline message queuing. Both talk to the same backend over REST and WebSocket.

Oracle stores structured user data, a sharded MongoDB cluster stores communication documents, and Redis handles caching and temporary state. Docker provides a reproducible environment for both local development and the production VM, where a scripted, health-checked deployment (deploy.sh) and a Let’s Encrypt/Nginx TLS gateway make the system reachable over a VPN.

NexusTeam.Shared supplies the common DTOs, models, enums, and realtime contracts that keep server and clients consistent. The next chapter describes how these architectural components are implemented in the backend and client applications.

4. Backend and Client Implementation

Backend Implementation Overview

The implementation follows the architecture described in Chapter 3. The system is divided into a server application, a shared project, a database seeding application, and two clients: the web client (primary) and the WPF client (optional). The backend is implemented in C# with ASP.NET Core 8 and provides the business logic, authentication, data access, REST API, and realtime communication. The web client is implemented in vanilla HTML/CSS/JavaScript; the WPF client follows MVVM. Both consume the same backend.

The implementation is organized around separation of responsibilities, dependency injection, and reusable services.

Concretely, the REST surface the controllers in §3.6 expose looks like this:

Area	Endpoints
Authentication	*POST /api/auth/register, POST /api/auth/login, POST /api/auth/refresh, POST /api/auth/logout*
Users	*GET /api/users, PUT /api/users/profile, GET/PUT /api/users/status, POST /api/users/avatar/upload, GET /api/users/avatar/{userId}*
Chats	*GET /api/chats, GET /api/chats/{id}, POST /api/chats, PUT/DELETE /api/chats/{id}, GET /api/chats/{id}/messages, POST /api/chats/{id}/messages*
Messages	*POST /api/chats/{id}/messages/forward, GET /api/messages/{chatId}/search, reaction, participant, pin and leave routes under /api/chats*
Attachments	*POST /api/attachments/upload, GET /api/attachments/download/{id},*

	thumbnail/{id}, message/{id}, PUT/DELETE /api/attachments/{id}
Preferences	*GET /api/preferences, PUT /api/preferences*
Chat folders	*GET/POST /api/folders, GET/PUT/DELETE /api/folders/{id}*
Calls	*POST /api/calls/history, GET /api/calls/history/{chatId}, GET /api/calls/history/missed/unseen, POST /api/calls/history/{id}/seen, GET /api/calls/ice-servers*
Device lock	*GET /api/device-lock/status, POST /api/device-lock/enable, PUT /api/device-lock, POST /api/device-lock/disable/unlock/lock/activity/forgot*
Health	*GET /health, GET /health/ready, GET /health/live*

Every authenticated endpoint expects a bearer JWT in the Authorization header; the server also publishes a Swagger/OpenAPI document in development so this surface can be explored and exercised interactively rather than only read about. Request and response bodies are plain JSON, serialized through the source-generated NexusTeamJsonSerializerContext mentioned in §4.18 for both the REST and WebSocket paths, which keeps one serialization configuration — not two — responsible for how every DTO and contract turns into bytes on the wire.

Backend Project Structure. The backend lives in NexusTeam.Server, with the main functional areas: Controllers, Services, Repositories, Middleware, Validators, Configuration, Data (Oracle/Mongo contexts), and Extensions (DI/app-builder wiring).

Dependency Injection. Interfaces are registered with the ASP.NET Core DI container instead of being constructed inside controllers or services — for example IAuthService, IChatService, IMessageService, IUserRepository, IChatFolderRepository, IWebSocketConnectionManager, and IResourceAuthorizationService.

Authentication, Users, and Core Domain Implementation

The main authentication components are AuthController, AuthService, JwtTokenService, RefreshTokenService, SessionService, and BcryptPasswordHasher.

JWT and Refresh Tokens. The access token authenticates subsequent requests; refresh tokens (stored in Redis with a 7-day TTL and individually revocable) maintain longer-lived sessions without repeated logins. AuthService handles the authentication workflow while JwtTokenService handles token creation, so the token-generation mechanism can change without touching the rest of the workflow.

Password Hashing. Passwords are protected with BCrypt, encapsulated in BcryptPasswordHasher. Hash parse/verify failures are caught and treated as a failed verification rather than an unhandled exception. The flow: Registration → BcryptPasswordHasher.Hash → users.password_hash; Login → BcryptPasswordHasher.Verify → issue tokens.

User Management. User functionality is implemented by UsersController together with IUserRepository, IUserStatusService, and IAvatarService; there is no separate UserService. The controller handles user listing/search, profile updates, status operations, and avatar upload/retrieval.

Chat Implementation. ChatsController, ChatService, and MongoChatRepository implement: creating chats, retrieving a user's chats, modifying chat information, and managing participants — with group deletion restricted to the chat owner and every operation checked against IResourceAuthorizationService.

Message Implementation. MessagesController, MessageService, and MongoMessageRepository implement messaging. The Message model carries an identifier, chat and sender identifiers, content, status, timestamps, optional reply/forward snapshot fields, an isDeleted/isSystem flag, attachments, and reactions (see the domain model in Figure 3).

Sending Messages. A message travels from client input through validation and authorization to persistence and realtime broadcast to the other participants.

```
User composes message → client REST call (POST /chats/{id}/messages)
   → MessagesController → validation (FluentValidation) → resource-authorization check
   → MessageService → MongoMessageRepository.Insert
   → WebSocketConnectionManager broadcasts MessageSentContract to participants
```

Figure 16 — Message Send Flow.

I uploaded the updated presentation to GitHub. I updated the stickers and added text about calls, wallpapers, forwarding, document previews, and so on.EditMessageRequest follows the same layered flow: authorization check (sender only) → MessageService.Edit → repository update (sets editedAt) → corresponding edit event broadcast. Deletion is a soft delete (isDeleted = true) followed by a corresponding delete event broadcast, so history is preserved for audit purposes.

Message History. Message history is stored in MongoDB and retrieved through MongoMessageRepository with cursor-based pagination (50 messages per page), independent of the realtime connection: WebSockets deliver current events, the database provides persistent history, and a client combines both to reconstruct a full conversation view.

Attachment Implementation. File handling is a dedicated slice: AttachmentsController, AttachmentService, MongoMessageAttachmentRepository, and attachment DTOs/models. AttachmentService writes uploaded files (and image thumbnails) to the server file system and stores only metadata (path, size, MIME type, attachment type) in MongoDB, up to the 100 MB per-file limit.

Chat Folder Implementation. Chat folders are a separate feature rather than being embedded in ChatService: ChatFoldersController, ChatFolderService,

MongoChatFolderRepository, plus their DTOs/models, keep folder organization independent of the underlying chat entities it references.

User Preferences. PreferencesController and its service handle theme, language, notification, privacy, and chat-display settings, stored per-user in MongoDB's user_preferences collection (with a 180-day inactivity TTL). Preferences are kept separate from account identity so they can evolve without touching authentication or user management.

Generated Image Implementation. GeneratedImagesController, GeneratedImageService, MongoGeneratedImageRepository, and the related DTOs/models let a user generate an image from a prompt inside a chat; the result is stored and referenced like any other attachment.

Realtime and Voice Implementation

The main server component is WebSocketHandler (a Middleware component, §3.23), which accepts the /ws upgrade, authenticates the connection, registers it with WebSocketConnectionManager, and then dispatches incoming envelopes to the right handler and outgoing broadcasts to the right recipients.

WebSocket Contracts. NexusTeam.Shared defines the WebSocket contracts carried inside the envelope (§3.15):

- ChatMessageContract — a new message was created.
- WebSocketMessageType.EditMessage with its payload — an existing message's content changed.
- WebSocketMessageType.DeleteMessage with its payload — a message was soft-deleted.
- TypingIndicatorContract — a participant started/stopped typing.

- MessageDeliveredContract / MessageReadContract — delivery and read receipts.
- UserJoinedContract / UserLeftContract — chat membership changes.
- UserStatusChangedContract — a presence/status update.
- Call-signaling contracts (CallSdpOfferContract, CallSdpAnswerContract, and ICE/hang-up equivalents) — WebRTC negotiation for voice calls (§3.21).
- ErrorContract — a realtime-channel error, including rate-limit rejections (RateLimitErrorPayload).

All payloads are registered with the source-generated NexusTeamJsonSerializerContext for fast, allocation-light (de)serialization.

Realtime Typing Indicators. When a user starts typing, the client sends a TypingIndicatorContract over the WebSocket; the server relays it to the other participants of that chat. Because typing state is transient, it is never written to MongoDB.

Message Delivery and Read Status. Separate contracts track the message lifecycle: Message sent → Message delivered (MessageDeliveredContract) → Message read (MessageReadContract), letting the UI show per-message delivery/read ticks.

Presence and Status. UserStatusService and the background PresenceTrackingService implement presence, backed by Redis. PresenceTrackingService periodically snapshots connected users via IWebSocketConnectionManager.GetConnectedUserIds(), refreshes LastSeenAt, and restores Online status for a user who is still connected but was marked Offline by a stale timeout — cancellation exits immediately rather than waiting out the retry delay.

```
Connection opened → status = Online (Redis, TTL 5m, refreshed on activity)
Connection closed → status = Offline after grace period → UserStatusChangedContract broadcast
```

Figure 17 — Presence State Flow.

WebSocket Connection Management. WebSocketConnectionManager maintains the user ↔ connection relationship so services can target a user without touching raw sockets. AddConnection rejects a duplicate connection ID before updating secondary indexes, so one connection ID can never end up registered under two users — important once reconnect races or multiple server nodes are involved.

WebSocket Authentication. WebSocket connections are not trusted by default. After the upgrade, the server expects authentication (a JWT) within a limited window; if it doesn't arrive in time, the connection is terminated, preventing unauthenticated connections from lingering.

Call Implementation. Voice functionality has two parts. Voice messages use the same attachment pipeline as files. Live web calls use WebSocket call-control/signaling contracts plus browser WebRTC for audio/video and screen sharing. The WPF client uses CallService with WebSocket call messages and NAudio capture/playback. CallsController and CallHistoryService record and retrieve call outcomes, while coturn provides TURN relay support for web calls in production.

Security, Reliability, and Operations Implementation

RateLimitService, backed by Redis counters (NexusTeam:ratelimit:{userId}:{endpoint}), throttles sensitive endpoints — five login attempts per five minutes per identifier, and roughly sixty messages per minute per user — returning a RateLimitErrorPayload (via ErrorContract/HTTP 429) when a caller exceeds its limit.

Health Checks. The server exposes /health, /health/ready, and /health/live, checking connectivity to Oracle, MongoDB, and Redis plus basic resource availability. The production deployment script polls these endpoints (via the

nexusteam_server/nexusteam_web container health checks) before cutting traffic over to a new release, and rolls back automatically if they never turn healthy.

Device Lock Implementation. DeviceLockController and DeviceLockOptions/DeviceLockOptionsValidator implement optional device binding: a user enables device lock with a PIN, which is hashed together with a server-side pepper (DeviceLockOptions.PinPepper) rather than stored or compared in plaintext. IUserDeviceService tracks known devices and their access state; DeviceLockMiddleware then rejects any request outside /api/auth, /api/device-lock, and /health that doesn't carry a currently unlocked device ID, independent of whether the JWT itself is valid.

Call History Implementation. CallsController (/api/calls) complements the WebSocket call-signaling contracts (§4.24) with REST endpoints backed by ICallHistoryService: recording a call's outcome (completed/rejected/missed/failed), retrieving a chat's recent call history, listing a user's unseen missed calls, and marking a missed-call entry as seen. Every endpoint re-checks chat membership through ChatService before returning history, so call history follows the same authorization rules as messages.

Client Implementation: Web and WPF

Web client (web/): index.html (structure), styles.css (presentation), app.js (application logic — auth, chat state, WebSocket handling, attachments, voice recording, folders, themes, notifications, image generation).

WPF client (NexusTeam.Client/): Views, ViewModels, Services, Infrastructure, Converters, and Helpers, following MVVM.

WPF Client Views and ViewModels. Twenty-one XAML views/windows/controls are included in the WPF client, with 26 ViewModel classes including supporting item and selector ViewModels. via CommunityToolkit.Mvvm. The main pairs:

View	Purpose	ViewModel
MainWindow	Application shell and navigation host	*MainWindowViewModel* — app state, navigation
WelcomeView	Landing/welcome screen	*WelcomeViewModel*
LoginView	Credential entry	*LoginViewModel* — authentication logic
RegisterView	Account creation	*RegisterViewModel* — registration logic
ChatView	Chat list and folders	*ChatViewModel* — chat list and management
ConversationView	Message thread for the open chat	*ConversationViewModel* — message display and sending (each message is a *MessageViewModel*)
SettingsView	Preferences and account settings	*SettingsViewModel*
GeneratorView	AI image generation	*GeneratorViewModel* — prompt handling and result display
FilesListView	Shared-files browser for a chat	*FilesListViewModel*
ImagesGridView	Generated-image gallery	*ImagesGridViewModel*
CodePreviewWindow	Syntax-aware code attachment viewer	—
TranslateWindow	On-demand message	*TranslateWindowViewModel*

	translation	
CreateChatDialog	New chat/group creation	*CreateChatDialogViewModel*
CreateFolderDialog	New chat-folder creation	*CreateFolderDialogViewModel*
— (call UI within ConversationView)	Call lifecycle controls	*CallViewModel — live audio call interface and management*
— (attachment rows)	Attachment presentation and supporting chat-folder/message UI	*AttachmentViewModel, ChatFolderViewModel, plus supporting utility views/view models*

MessagingService (WPF). MessagingService coordinates realtime communication with the backend on the WPF client: retrieving, sending, and editing messages, and dispatching incoming WebSocket events to the right ViewModel through the MessageBus.

AuthenticationService (WPF). AuthenticationService manages client-side authentication state and talks to the backend auth endpoints, so LoginViewModel never needs to know how tokens are created or how HTTP requests work — it calls the service and reacts to the result. This keeps the authentication implementation independently changeable from the login UI.

Credential Storage (WPF). CredentialStorageService uses LiteDB for local, encrypted client-side storage, kept separate from authentication itself so the app distinguishes "authenticated with the server" from "has stored client-side credentials." The LiteDB database, kept under %LocalAppData%\NexusTeam\Data\, holds more than just credentials — encrypted user credentials, cached user data (so a profile doesn't have to round-trip to the server every time it's displayed), the offline message queue (§4.32),

application settings, and session tokens all live there, which is why the service is treated as its own component with its own responsibility rather than folded into **AuthenticationService:** a bug in how tokens are cached shouldn't be able to corrupt the offline queue, and vice versa, because the two concerns share a storage engine but not a code path.

Offline Message Queue (WPF). OfflineMessageQueue temporarily retains messages that cannot be sent immediately because the connection is unavailable.

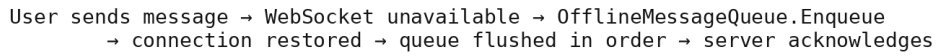


Figure 18 — Offline Message Queue Flow.

Reconnection Strategy. Both clients reconnect automatically after a WebSocket interruption, re-authenticating and re-subscribing on success so realtime communication resumes without a manual restart.

WPF Startup Sequence and Performance. The WPF client is launched with the server's address and port as command-line arguments (NexusTeam.exe localhost 5251, or the equivalent dotnet run localhost 5251 from source), rather than a hardcoded server location, so the same build can point at a local development server, a LAN server, or — since the arguments accept any reachable host — the production VM's address. Startup itself follows a fixed sequence: parse the command-line arguments, configure logging, configure exception handling, initialize the dependency-injection container, build and start the generic host, attempt to restore a previous session from CredentialStorageService (§4.31), navigate to either the conversation view or the welcome/login view depending on whether that restore succeeded, and finally show the main window. Because every step up to "show the main window" happens before the user sees anything, the client keeps I/O off the UI thread throughout — all network

operations are asynchronous — and applies list virtualization to the message and chat lists so that a long conversation history doesn't have to be fully realized into UI elements just to scroll through it. Frequently accessed data (the current user's profile, open chats) is cached rather than re-fetched on every navigation, and disposable resources (WebSocket connections, database handles) are disposed deterministically rather than left for the garbage collector, which matters for a long-running desktop process more than it would for a short-lived request handler.

File Attachment Client Implementation. FileAttachmentService (WPF) separates file selection, validation, and upload from the rest of the messaging logic, so the UI can request an attachment operation without implementing the upload protocol itself. The web client's app.js implements the equivalent flow directly against the REST attachment endpoints.

Avatar Implementation. AvatarService abstracts loading and managing avatar data on the WPF client, keeping image-related operations out of unrelated components.

Theming and Styling (WPF). The WPF client ships a dark theme by default and a light theme as an alternative, defined in Themes/DarkTheme.xaml and paired with shared visual resources in Themes/Styles.xaml. Visual formatting logic lives in Themes/Converters.xaml, which is where the value converters introduced in §5.29 are registered as application-wide resources rather than being instantiated per-view — one UserStatusToBrushConverter instance, for example, backs every presence dot in the app. Beyond the converters, the styling layer is responsible for a consistent color scheme, custom window chrome (so the app doesn't look like an out-of-the-box WPF window), smooth transition animations, and a layout that stays legible as the main window is resized.

Translation Implementation. TranslationService (WPF) calls the DeepL API to translate message content, implemented as a separate feature rather than being baked

into the conversation ViewModel, so it can change or be replaced independently. The current web client has no corresponding translation workflow.

Image Generation Client.
GeneratorView/GeneratorViewModel/ImageGeneratorService (WPF) and the equivalent web-client flow in app.js both call GeneratedImagesController to generate an image from a prompt and attach it to the conversation.

Navigation (WPF). NavigationService centralizes navigation between views so individual ViewModels don't need to know the structure of the whole UI.

Error Handling. The backend has centralized ExceptionHandlingMiddleware, mapping known exceptions to the right HTTP status (UnauthorizedException → 401, NotFoundException → 404) without leaking internal exception messages in the public response Detail field. The WPF client has a dedicated ErrorHandlingService; the web client surfaces errors inline in the UI. The WPF error-handling strategy is deliberately layered rather than a single try/catch at the top of the call stack: service-level errors are caught and logged at the point they occur (so the log has the actual context, not just "something failed somewhere"), a generic user-friendly message is what actually reaches the UI (so a database connection string never ends up on screen), a dedicated error-notification event lets any interested ViewModel react without every component needing its own error-handling code, transient errors get automatic retry rather than immediately surfacing to the user, and — for the specific case of a lost connection — the offline queue (§4.32) absorbs the failure instead of treating it as an error at all. Errors are also classified by severity (Info, Warning, Error, Critical) so that the UI, and a future monitoring pipeline (§3's scalability discussion notes this as a current gap), can distinguish "worth telling the user" from "worth paging someone about" rather than treating every caught exception identically.

```
Database/Service error → Exception → ExceptionHandlingMiddleware → mapped HTTP status/body
                                                                  → client ErrorHandlingService/UI
```

Figure 19 — Error Handling Flow.

Logging. Serilog provides structured logging for debugging, diagnosing connection problems, monitoring behavior, and identifying errors, writing to console in development and to file in production, with request/response and WebSocket events logged via RequestLoggingMiddleware. Log levels are used deliberately rather than as an afterthought: Error is reserved for exceptions and critical issues, Warning covers validation failures and rate-limit rejections (things that are expected to happen occasionally and shouldn't page anyone, but are worth counting), Information covers ordinary API requests and WebSocket events, and Debug — enabled only in development — covers detailed execution flow that would be too noisy to keep in a production log. The current setup covers logging itself well but stops short of full observability: there is no metrics pipeline collecting request latency, WebSocket connection counts, message throughput, or cache hit/miss ratio into a queryable time series, and no dashboard for database query time, connection-pool utilization, or host-level CPU/memory/disk/network figures. Structured log output and the health-check endpoints (§4.26) are what the project currently has in place; a metrics backend (Application Insights or an ELK-style stack were the two considered options) and dashboards built on top of it are recorded as future work rather than partially built, so that this documentation doesn't overstate what can currently be inspected in production versus what would need to be added first.

Database Seeding. NexusTeam.DbSeeder initializes Oracle, MongoDB, and Redis structures on first startup and seeds four demo accounts (Vlad, Sofia, Hakan, Anna, shared demo password) — recreated on every startup, which is convenient for a VPN-only demo but must be disabled before the app is exposed to untrusted users (§3.35, §8.17).

Implementation of Shared Models and DTOs. NexusTeam. Shared contains the common models (§3.17) and DTOs used across components — request types for login, registration, sending/editing messages, creating chats, uploading attachments, updating preferences, and generating images. Keeping these as a physically separate project rather than, say, duplicated classes in the server and each client, has a concrete build-time consequence worth spelling out: NexusTeam.Server, NexusTeam.Client, and the type definitions consumed by the web client's JSON handling all reference the same compiled assembly for these types (the web client, being JavaScript, works from the JSON shape rather than the C# types directly, but that JSON shape is itself generated from the shared DTOs via NexusTeamJsonSerializerContext). A field renamed on a shared DTO is therefore a compile error in every C# project that used the old name, not a silent runtime mismatch discovered only when a request fails — which is precisely the integration-risk mitigation described in §7's discussion of shared contracts and team coordination, made concrete at the level of an individual class.

Client-Server Communication. Clients talk to the backend through two mechanisms: REST, for authentication, loading data, and modifying resources; and WebSocket, for the realtime events and call signaling described in §3.14–3.16 and §3.21.

Web Client Implementation. The web client is implemented with plain HTML, CSS, and JavaScript, with the main application logic in app.js: authentication, WebSocket handling, chat and message rendering, attachments, voice-message recording, folders, themes, and image generation.

Web Client Deployment. The web client is packaged with its own Dockerfile and served through Nginx, which also proxies /api and /ws to the backend container.

```
Browser → (dev) http://localhost:8080  /  (prod) https://<PUBLIC_IP>
        → Nginx (nexusteam_web, static files + reverse proxy)
        → NexusTeam.Server (nexusteam_server, REST + WebSocket)
```

Figure 20 — Web Client Deployment Flow.

In production, an additional Nginx/Certbot TLS gateway sits in front of nexusteam_web and terminates HTTPS with a Let's Encrypt certificate (see §3.28, §8.17).

Implementation Quality, Challenges, and Limitations

Interfaces are used for major services and repositories; dependencies come through the DI container rather than being constructed directly; and shared contracts in NexusTeam.Shared keep the server and both clients in sync without duplicating definitions.

Implementation Challenges. The first major challenge is realtime communication: WebSockets need connection lifecycle management, authentication, reconnection, message routing, and state synchronization, well beyond a plain HTTP request. The second is integrating three different storage technologies with consistent behavior. The third is supporting two clients from one backend — a change to the API or to a shared contract can affect the server and both clients at once. A fourth, newer challenge is voice calling: WebRTC signaling, NAT traversal, and a TURN relay add real-time media concerns on top of real-time messaging, and are implemented differently in the two clients: browser WebRTC for the web client and NAudio over WebSocket for WPF. Finally, going from "runs in Docker on a laptop" to "runs on a production VM behind a VPN with a trusted certificate" introduced its own set of concerns: secret management, health-checked cutover, and rollback.

Implementation Limitations. Automated test coverage should be expanded to cover more services, repositories, and client components (Chapter 6). The web and WPF clients use different live-call media implementations, so their call interfaces and capabilities are not identical. The demo-account seeder is unsuitable for a public deployment as-is. Additional observability, performance monitoring, and automated integration/E2E testing in CI would further strengthen the implementation. These

limitations do not prevent the system from demonstrating the intended software engineering concepts, but they remain areas for further improvement (Chapter 9).

Summary. The implementation follows the layered, modular architecture from Chapter 3. The ASP.NET Core backend has dedicated controllers, services, repositories, middleware, authentication, rate limiting, health checks, and infrastructure services; Oracle, MongoDB, and Redis are used according to the type of data and its performance requirements. Realtime communication runs through WebSockets and a set of shared contracts, supporting messaging, typing indicators, presence, message status, and voice communication (voice messages on both clients, live audio/video calls and screen sharing on the web client, and live audio calls on the WPF client). The web client is the primary, framework-free browser UI; the WPF client is an optional MVVM-based Windows companion with its own dedicated services for authentication, messaging, calls, attachments, avatars, translation, image generation, navigation, and offline communication. Separation of responsibilities, dependency injection, shared contracts, repository abstraction, and dedicated services keep the implementation modular and extensible — and now also deployable, end to end, to a real production VM.

5. User Interface

Design Goals and Structure

The user interface is the main point of interaction between users and NexusTeam. It provides access to messaging, user management, file sharing, chat organization, calls, settings, and supporting features while keeping the core communication workflow simple.

NexusTeam provides two client interfaces:

- the web client — a responsive browser-based application implemented in HTML, CSS, and JavaScript, and the primary way users access the product;
- the WPF desktop client — an optional, Windows-only native application, used mainly for development and voice calls.

The two clients use different technologies and different visual implementations, but they connect to the same backend and expose the same central communication platform, so the concepts a user learns in one transfer to the other.

UI Design Goals. A messaging application is used frequently, in short bursts, so common actions should not require navigating through unnecessary screens. The main goals are simplicity (no unnecessary complexity in opening a chat, sending a message, or switching conversations), consistency (shared concepts and visual language across both clients), realtime feedback (typing, delivery, presence, all visible without a manual refresh), and accessibility of the functionality people use every day.

User Interface Structure. Both clients share the same conceptual layout.

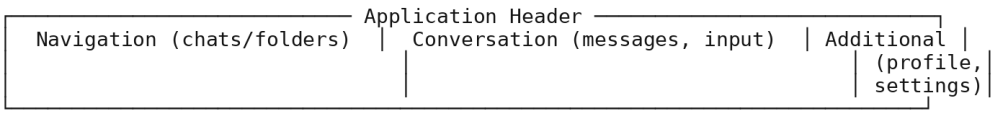


Figure 21 — UI Layout Structure.

A Typical Session. A new user opens the web client at http://localhost:8080 (or the production URL) and lands on the welcome screen. They choose registration, enter the required fields, and submit the form. Client-side validation provides immediate feedback for obviously invalid input, while the server-side FluentValidation rules (§3.24) remain the authoritative validation. After successful registration, the client stores the issued JWT and refresh token (§4.5, §4.31) and opens the main application.

From the main window, the user sees an empty chat list and searches for another user (FR-05) to start a conversation. Creating a chat opens the create-chat dialog (§5's Dialogs discussion), and once a chat exists, the conversation view becomes the focal point of the screen: the web client's responsive layout puts the chat list and the open conversation side by side on a desktop-sized viewport, while on a phone-sized viewport only one of the two is visible at a time, with a back action returning from the conversation to the list (§5's Responsive and Adaptive Considerations discussion). Typing a message shows the other participant a typing indicator within roughly the round-trip time of a WebSocket frame; sending it updates the message list optimistically on the sender's own screen before the server's acknowledgment even arrives, then reconciles that optimistic state against the server's actual response — a message that fails to send (a dropped connection, a rejected request) is visually distinguished from one that succeeded, rather than silently looking identical to a successful send until the user notices the recipient never replied.

Attaching a file follows the same "acknowledge immediately, reconcile against the server afterward" pattern: selecting a file shows a local preview before the upload to the attachments endpoint (§4.13) completes, so the user isn't staring at a blank composer

while a large file uploads. Generating an image or requesting a translation both follow a visibly different pattern, appropriately, since both involve a round trip to an external or AI-backed service that can take noticeably longer than a database write — the UI shows a loading state for the duration of the request rather than either blocking the rest of the interface or pretending the operation is instantaneous. Finally, opening settings takes the user out of the conversation-centric part of the UI entirely, into a dedicated view for theme, notification, and privacy preferences (§4.15) — a deliberate separation (§5's Settings Interface discussion) so that the screen someone uses dozens of times a day and the screen they use rarely don't compete for the same visual space.

Core Communication Interface

The first interaction is the welcome and authentication flow: welcome screen → login or registration → authentication feedback. On the WPF client this is three distinct views rather than one screen wearing different modes: WelcomeView carries the application's branding along with a login button, a register button, and — since the WPF client persists sessions locally (§4.31) — an attempt at silent session restoration before the user has to interact with anything at all. LoginView narrows to exactly what authentication needs: username/email, password, a "remember me" checkbox, a login action, and a link to registration. RegisterView mirrors it with the fields registration actually requires — username, email, password, and display name — plus a link back to login for someone who reaches it by mistake. Splitting these into three views rather than one collapses-and-expands form keeps each view's XAML focused on one job and keeps LoginViewModel and RegisterViewModel from having to coordinate shared visibility state between two workflows that otherwise have almost nothing in common besides both preceding authentication.

Login Interface. The login screen authenticates the user with the minimum required interaction: username/email input, password input, a login action, and navigation to registration.

Registration Interface. The registration form collects the required account information and shows validation feedback if it doesn't satisfy the backend's rules: enter information → validate → submit → confirmation or error.

Main Application Window. After authentication, the main window becomes the central navigation point, giving access to conversations, chat folders, and user/profile information.

Chat List. Each chat entry shows the participant/group name, avatar, latest-message preview, and conversation state (unread count, mute, pinned).

Chat Folders. As the number of conversations grows, a flat chronological list becomes hard to navigate, so NexusTeam provides chat folders that let users group conversations by their own criteria, with dedicated creation and management UI. This also demonstrates separating content organization (folders) from the underlying conversation data (chats).

Conversation View. The conversation view is the central interface: a conversation header, participant information, and message history, plus the controls for sending new content.

Message Presentation. Messages are shown as individual elements, visually distinguishing the current user's messages from received ones so direction is clear without reading every sender name. Elements can carry text, timestamps, reply/forward context, attachments, and reactions.

Message Actions. Depending on the message and the user's permissions, available actions include editing, deleting, viewing attachments, reacting, replying, and copying content.

Message Input. The message input keeps sending a message as the immediately accessible primary action, while attachments, voice recording, and the emoji picker stay one tap/click away rather than in the primary path.

Typing Indicators. Typing information appears as realtime feedback without a refresh, and is presented as part of the current conversation state rather than as a permanent message, since it is inherently transient.

Message Delivery and Read State. The UI distinguishes sent, delivered, and read states, giving users context about whether a message reached its recipient without that status becoming the primary content of the conversation.

User Presence. Presence updates (online/away/offline/DND, last seen) are reflected in realtime, which is especially useful in direct conversations to gauge whether the other participant may currently respond.

User Profile. The profile view shows username, avatar, profile information, and status, and lets a user edit their own.

Settings Interface. Settings live in a dedicated view, separated from the conversation interface so configuration doesn't interfere with everyday communication: theme, language, notifications, privacy, and chat-display preferences.

File Attachments. The attachment workflow: select attachment → review/process file (thumbnail for images, type detection) → upload → attach to the message being composed.

Images. Images get dedicated presentation: an image viewer dialog for inspecting a photo at full size without leaving the conversation context.

Files List. A dedicated files view lists shared files independently of the conversation, useful when a user needs to locate a shared file without scrolling — a content-oriented view that complements the conversation's chronological one.

Emoji Picker. An emoji picker is integrated into message composition so adding an emoji doesn't require leaving the current conversation or remembering Unicode input.

Visual Language. Both clients lean on the same small set of visual conventions rather than reinventing one per screen. Status is always communicated the same way regardless of which feature it appears in — a colored presence dot for a user, a colored badge for an unread count, a colored tick set for message delivery/read state — so a user who has learned what green means in one place doesn't have to relearn it in another. The WPF client formalizes this through its theme resources: a dark theme is the default, a light theme is available as an alternative, and both are built from the same underlying style and converter set discussed earlier in this chapter, rather than as two independently maintained visual designs that could quietly drift apart. The web client follows the same conventions through its own CSS, so a screenshot of a chat list from either client is recognizably "NexusTeam" rather than looking like two unrelated products that happen to share a login system.

Extended Features and Cross-Platform Consistency

The WPF client contains 21 XAML views, windows, and controls (§4.28's table). The desktop interface separates several actions into dedicated windows or dialogs, while the web client keeps related actions within its continuous browser interface. TranslateWindow and CodePreviewWindow are examples of focused desktop surfaces. The underlying data and functionality remain shared through the same backend; the main difference is how the interface presents those functions on each platform.

Both clients let a user record and send a voice message: the web client uses the browser MediaRecorder API; the WPF client uses NAudio. Voice recording is handled by its own service on each client, kept separate from the general messaging code path.

Voice Calls.Live call functionality is available through the web client and the WPF client, with different media implementations. The web client establishes audio/video

calls with browser WebRTC and supports screen sharing; the WPF client provides live audio through CallService, WebSocket call messages, and NAudio. Both use the shared backend call protocol and call-history subsystem.

Image Generation Interface.
GeneratorView/GeneratorViewModel/ImageGeneratorService (mirrored in the web client's UI) let a user enter a prompt and view the generated image once it's ready.

Translation Interface. The WPF client provides a dedicated TranslateWindow and translation action backed by DeepL. The current web client does not expose the translation feature.

Dialogs. Both clients use dedicated dialogs for create-chat, create-folder, edit-group, and image-viewer interactions, keeping those short-lived tasks out of the main conversation view.

Code Preview. A code-preview surface (CodePreviewWindow/CodePreviewControl on WPF, an equivalent in-message preview on the web client) renders code attachments with language-aware formatting rather than as plain paragraph text.

WPF UI Architecture. The WPF UI follows MVVM, as described in §3.19.

```
View (XAML)  ⇄ data binding ⇄  ViewModel (state + commands)  ⇄  Service  ⇄  Backend
```

Figure 22 — WPF Data Binding Flow.

Data Binding. WPF data binding connects UI elements to ViewModel properties, so state changes (a new message, a changed status, a newly selected chat) update the interface automatically instead of requiring manual UI manipulation. A family of dedicated value converters keeps that binding declarative rather than pushing formatting logic into the ViewModels: BooleanToVisibilityConverter and InverseBooleanToVisibilityConverter show or hide elements from a boolean flag, BooleanToAlignmentConverter and BooleanToBackgroundConverter give a sent-by-me

message a different alignment and background than a received one (§5.11's visual distinction is implemented here), UserStatusToBrushConverter and UserStatusToTextConverter turn a UserStatus enum value into a presence dot color and label, CountToVisibilityConverter hides an unread-count badge when it's zero, FirstCharacterConverter derives an avatar-placeholder initial from a display name, and NullToVisibilityConverter/NullToBooleanConverter/StringToVisibilityConverter handle the common "only show this if that data actually exists" case throughout the views. Keeping this logic in converters rather than in the views or ViewModels means a XAML designer can wire up a new binding without duplicating formatting rules, and a formatting rule only has to be fixed in one place.

Commands and User Actions. User actions are wired to ViewModel commands (RelayCommand) rather than invoking application services directly from the UI — login, registration, selecting a chat, sending a message, and so on.

Web Client User Interface. The web client — HTML, CSS, and JavaScript — provides access to the core messenger functionality in any modern browser, with no installation required.

Web UI Structure. The web client follows the same conceptual model as the WPF client, communicating with the backend over REST and WebSocket, but organizes its own DOM/state management directly in app.js rather than through an MVVM framework.

Responsive and Adaptive Considerations. The web client has to handle a much wider range of viewport sizes than the WPF client's predictable Windows window. It therefore uses responsive layout: a side-by-side chat-list/conversation layout on desktop-sized viewports, and a full-screen, single-pane navigation (list → conversation → back) on narrow viewports (≤768px), where showing both panes at once would leave neither usable. The 768px threshold is not an arbitrary round number — it sits at the

conventional boundary between a phone held in portrait orientation and a small tablet or a narrow desktop window, chosen so that the layout switch happens where it actually matches a change in how much horizontal space a user realistically has, rather than at a number that merely looked tidy. Between the two extremes, the same CSS rules apply continuously: text reflows, touch targets stay large enough to tap reliably rather than shrinking to a desktop-appropriate size, and the message composer stays pinned to the bottom of the viewport on mobile so it remains reachable without scrolling, which matters more on a phone than on a desktop where the whole window is usually within easy reach already.

UI Consistency Across Platforms. Supporting two clients risks giving users two different interaction models. NexusTeam reduces this by keeping the same underlying concepts — users, chats, messages, folders, presence — visible and named consistently in both clients, even though the visual implementation differs.

Feedback and Error States. The interface surfaces invalid login credentials, invalid registration data, failed message delivery, and connection problems as clear, in-context feedback rather than silent failures. The specific wording of these messages was deliberately kept generic rather than exposing the backend's internal reasoning — a failed login says something to the effect of "invalid username or password" regardless of which of the two was actually wrong, which is a small, easy-to-miss detail that is nonetheless a real security property (§3.29's authorization discussion): a more specific message ("no such user" versus "wrong password") would let an attacker enumerate valid usernames one login attempt at a time, so the UI's error wording and the backend's ExceptionHandlingMiddleware (§4.39) are both, independently, designed around the same "don't leak more than necessary" principle rather than one of them being an accident that happens to align with the other.

Loading States. Loading indicators cover authentication, loading conversations, retrieving message history, and uploading attachments, so a slow network operation doesn't make the interface look frozen.

Usability Considerations.

Recognition instead of recall — frequent actions are visible controls, not commands to remember. Consistent placement — related controls are grouped for predictable interaction. Immediate feedback — actions get a visible response (a sent-message tick, a loading state, an error banner) right away.

Accessibility Considerations. Accessibility is addressed through sufficient contrast, readable text, recognizable controls, and keyboard navigation, with room for further systematic accessibility testing (§5.43). The web client uses standard HTML controls where possible, while the WPF client uses its own control templates and AutomationProperties. Neither client has yet undergone a complete accessibility audit covering screen readers, contrast-ratio verification against WCAG thresholds, and a full keyboard-only walkthrough of every workflow, so this remains a limitation (§5.43).

UI Security Considerations. Password fields are masked. Neither client displays raw access tokens or refresh tokens in the UI; both are handled behind the authentication/credential-storage services (§4.5, §4.31) rather than being exposed to view code.

UI and Backend Integration. The interface is closely connected to the backend but never touches its internals directly — every operation goes through the REST/WebSocket boundary described in Chapter 3, mediated on the WPF client by dedicated services and on the web client by app.js.

Usability Evaluation Approach. Beyond the automated and manual functional testing described in Chapter 6, the UI was evaluated with several concrete tasks: registering two accounts, exchanging messages, creating a group, organizing chats into a folder, sending

an attachment, and locating a previously sent file. The tasks were checked on both clients. This approach was narrower than a formal usability study with external participants, but it helped identify practical UI issues during development, including an early folder-creation flow that did not make multi-folder assignment sufficiently clear.

UI Evaluation

Feature coverage — the interface reaches nearly all backend functionality: there is no significant server capability (chats, folders, attachments, preferences, translation, image generation, calls) that lacks a corresponding UI entry point on at least one client. Modular structure — dedicated views, dialogs, ViewModels, and services per feature, so a change to how folders work, for instance, touches ChatFolderViewModel/CreateFolderDialog and the equivalent web-client code without rippling into unrelated screens. Two platforms, one concept model — a user who learns one client largely already knows the other, because both are built against the same domain vocabulary (chats, messages, folders, presence) rather than platform-specific reinterpretations of it.

Weighed against those strengths, the interface's current weak point is asymmetry rather than missing functionality: the WPF client differs from the web client in live-calling media implementation and provides the additional offline message queue, while the web client is ahead on reach (no installation and support for modern desktop and mobile browsers) and on responsive layout. Neither client is simply "better" — they currently serve slightly different roles (universal daily-driver vs. development/voice-call companion) more than they serve as fully interchangeable equals, which is itself a reasonable and explicit product decision rather than an oversight, but one worth stating plainly for anyone evaluating the project.

UI Limitations and Future Improvements. The UI could be further standardized between the two clients with a shared design system (colors, typography, spacing, icons,

interaction patterns). Accessibility testing could be expanded with automated tooling and dedicated user testing. The web client's responsive layouts could be refined further for very small or very large screens. Additional usability testing would help confirm that less-frequent functionality sits in the right place. Voice calls and AI-generated content would benefit from more detailed progress, error, and state indicators — and, most concretely, the web client can be improved further with more detailed live-call state and error indicators.

Summary. The NexusTeam UI provides access to the system's communication and supporting functionality through two client applications sharing one backend. The web client is the primary, browser-based interface, implemented in plain HTML/CSS/JavaScript with its own dedicated flows for authentication, messaging, attachments, folders, profiles, settings, voice messages, and image generation. The WPF client is an optional Windows companion built with MVVM, adding a native desktop experience and live voice-calling UI implemented with its own native media stack. Both are designed around simplicity, consistency, realtime feedback, and accessibility of frequently used functionality, and both keep the presentation layer independent of the underlying backend implementation.

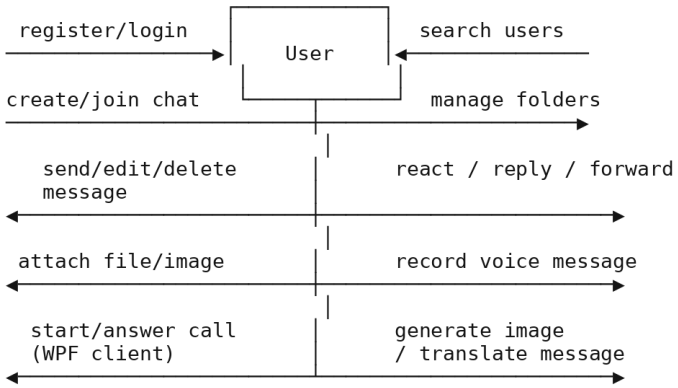


Figure 23 — Use Case Diagram.

6. Testing

Testing Strategy and Environment

Testing is essential for NexusTeam because the system spans a backend server, two client applications, three databases, realtime communication, authentication, and containerized infrastructure. A failure in one component can affect others — a change to a shared WebSocket contract can affect the server and both clients, and a database configuration problem can silently break authentication or messaging. The testing approach therefore covers the system at several levels: functional, integration, API, WebSocket, database, security, and end-to-end.

Testing Objectives.

- Verify implemented functionality against the requirements in Chapter 2.
- Identify defects before deployment, including before a production release to the VM.
- Verify communication between both clients and the backend.
- Verify correct database operations across Oracle, MongoDB, and Redis.
- Verify authentication, authorization, rate limiting, and device-lock behavior.
- Verify realtime WebSocket communication, including call signaling.

Testing Strategy. Testing follows a layered approach, from client-level checks through backend unit/integration testing up to full system and end-to-end coverage.

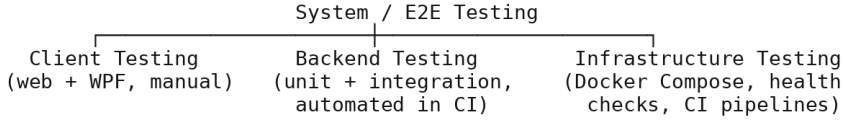


Figure 24 — Testing Strategy Layers.

The testing strategy uses several layers. Unit tests check isolated logic such as validators and service methods and can run quickly in CI. Integration tests check communication with databases and infrastructure components. API and WebSocket tests verify the server interfaces and realtime behavior, while end-to-end tests verify complete workflows across the system. The project currently contains 50 server unit-test files and 10 shared-project unit-test files, for 60 unit-test files in total (§6.41).

Test Environment. The test environment mirrors the development environment: NexusTeam.Server, the WPF NexusTeam.Client, the web client, Oracle Database, the MongoDB sharded cluster, and Redis, all runnable via Docker Compose. CI runs the same containers in GitHub Actions.

Test Data. Testing uses dedicated test accounts and conversations rather than real user data: valid user accounts, invalid registration data, incorrect passwords, direct conversations, group conversations, and — for E2E — the four documented demo accounts.

Functional and Component Testing

Functional testing verifies the operations defined in Chapter 2's functional requirements: registration, authentication, device lock, user management, chat creation, messaging, attachments, folders, presence, realtime events, voice messages/calls, call history, translation, and image generation.

A common functional-test pattern is used throughout the project: create an authenticated test context, perform the operation, and verify both the direct result and the resulting persisted state where applicable. For example, a message test can verify the HTTP response or WebSocket event and then check that the message was stored correctly. This pattern is applied to authentication, messaging, attachments, preferences, device lock, call history, and other tested features.

Authentication Testing.

- Valid registration — input: valid registration information; expected: a new account is created.
- Duplicate registration — input: an already-registered username or email; expected: rejected with a validation error.
- Valid login — expected: an access token and refresh token are issued.
- Invalid credentials — expected: login rejected, no token issued, and the failed attempt counted toward the rate limit.
- Token refresh — expected: a valid refresh token yields a new access token; a revoked or expired one does not.

Authentication Test Cases. Covered automatically by AuthControllerTests, AuthServiceTests, JwtTokenServiceTests, RefreshTokenServiceTests, and AuthenticationE2ETests, which exercise both normal authentication behavior and the invalid-input scenarios above end to end against a running server.

User Management Testing.

- Retrieving an existing user; searching for users; updating profile information; changing an avatar; attempting to access or modify another user's information (expected: denied).

Covered by UsersControllerTests, AvatarServiceTests, and AvatarE2ETests/ProfileAndPreferencesE2ETests.

Chat Testing. Create a direct conversation; create a group conversation; retrieve an existing conversation; add/manage participants; access a conversation as an authorized participant; attempt access as a non-participant (expected: denied). Covered by ChatsControllerTests, ChatServiceTests, and ChatAndFolderE2ETests.

Message Testing. Sending messages; retrieving message history (with pagination); editing messages (sender only); deleting messages (soft delete); message ordering; reactions. Covered by MessagesControllerTests, MessageServiceTests, and MessagingE2ETests.

Message Test Cases. Boundary cases include an empty message, a message at the 4000-character limit, a message one character over the limit, and an edit/delete attempt by a non-sender (expected: 403).

Realtime WebSocket Testing. Connection establishment; authentication; authentication timeout (connection closed if no auth arrives in time); message reception; message broadcasting to all participants except, where applicable, the sender; duplicate connection-ID rejection. Covered by WebSocketHandlerTests, WebSocketConnectionManagerTests, and WebSocketE2ETests.

WebSocket Test Cases. A representative case: two authenticated clients join the same chat; Client A sends a message; the test asserts Client B receives a ChatMessageContract with matching content within a bounded time window.

Typing Indicator Testing. Client A starts typing → Client B receives the typing-started event. Client A stops typing → Client B receives the typing-stopped event. The test also checks that a typing indicator does not persist as a stored message and does not linger after the sender stops.

Presence Testing. User A logs in; User B observes User A's status; User A changes status or disconnects; User B receives the corresponding update; the interface reflects the latest known state, including the PresenceTrackingService reconciliation described in §4.21.

Attachment Testing. Selecting a valid file; uploading a file; retrieving an attachment; attaching a file to a message; handling an invalid file (oversized, disallowed type);

handling a corrupted upload. Covered by AttachmentServiceTests, AttachmentsControllerTests, and BinaryDataE2ETests.

Chat Folder Testing. Creating a folder; renaming a folder; retrieving folders; adding conversations; removing conversations; deleting a folder without deleting the underlying chats. Covered by ChatFolderServiceTests, ChatFoldersControllerTests, and ChatAndFolderE2ETests.

Voice Message and Call Testing. Voice messages require testing recording, cancellation, upload, storage, delivery, playback, and invalid-input handling: User A records and sends a voice message; User B receives and plays it; a recording is cancelled before sending; microphone/permission errors are handled without exposing technical details; negative cases include an unavailable microphone, interrupted upload, unsupported audio data, and network disconnection during upload.

Call testing additionally covers call-signaling contracts (offer/answer/ICE, ringing/accept/reject/hang-up/busy), call-history recording and retrieval, and unseen-missed-call surfacing on reconnect. Covered by CallsControllerTests and DeviceLockE2ETests/E2E call-signaling coverage where applicable.

Database Testing.

Oracle — user creation, user lookup, uniqueness constraints (username/email), account updates, status check-constraint enforcement. **MongoDB** — document schema validation for chats/messages/attachments/preferences/folders/generated images, index usage, TTL expiry for preferences. Redis — session TTL/refresh, presence TTL and refresh-on-activity, rate-limit counter behavior, refresh-token revocation.

Database Consistency Testing. A functional workflow may span more than one store — a message workflow involves MongoDB persistence and realtime WebSocket state; a session involves Redis and JWT claims. Tests verify the system never exposes an inconsistent state to the user, e.g.: Message sent → persisted in MongoDB →

ChatMessageContract broadcast to connected participants → both reflect the same content and ordering.

API, Security, and Client Testing

REST endpoints are tested independently of the UI, so a defect can be attributed to backend or client: valid requests, invalid requests, authentication, authorization, and rate limiting. Covered broadly by the NexusTeam.Server.Tests/Controllers suite and ApiContractE2ETests.

API Validation Testing. Missing required fields; invalid identifiers; excessively long strings (e.g. message content over 4000 characters); invalid email formats; invalid credentials; a null participant list on chat creation (explicitly hardened per the project TODO.md).

Security Testing.

- **Authentication bypass —** access a protected endpoint without authentication; expected: 401.
- **Invalid token —** access with a malformed/expired token; expected: 401.
- **Resource authorization bypass —** a non-participant attempts to read or modify a chat/message/attachment they don't belong to; expected: 403/404, not a data leak.
- **Device lock bypass** — a request without a bound device hits an endpoint outside /api/auth, /api/device-lock, /health; expected: 401.
- **Rate limit —** exceed the login or message-send limit; expected: throttled with 429.

Security Test Cases. Covered by SecurityE2ETests, DeviceLockE2ETests, RateLimitServiceTests, JwtAuthenticationMiddlewareTests, and SecurityHeadersMiddlewareTests.

Client Testing. The WPF client is tested from the user's perspective and through its individual services: startup, authentication, navigation, chat selection, message sending, call handling.

WPF Client UI Testing. Example workflow: Start application → Login → Main window → select chat → send message → verify delivery. UI testing here is primarily manual, given the difficulty of automating WPF UI interaction.

Web Client Testing. The web client is tested independently because browser behavior differs from the WPF runtime: registration, login, chat loading, sending messages, realtime events, responsive layout at different viewport widths.

Cross-Client Testing. One of the most important system-level tests is communication between the two different clients.

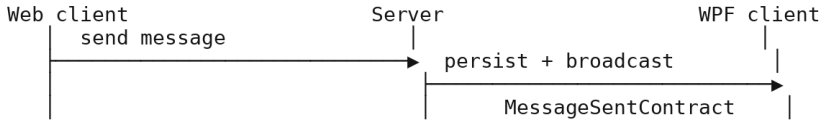


Figure 25 — Cross-Client Message Flow.

Reconnection Testing. Simulate a network interruption: the client detects the failure, the realtime connection is marked unavailable, the client attempts reconnection, the connection is restored, and realtime communication resumes without a manual restart.

Offline Message Testing. Establish a normal connection → disconnect → create a message → verify it is retained in the WPF client's OfflineMessageQueue → restore connectivity → verify the queue flushes in order and the server acknowledges each message.

Performance, Automated Testing, and Evaluation

Sending multiple messages in sequence; loading a large message history page; establishing multiple concurrent WebSocket connections; uploading attachments near the 100 MB limit; repeated API requests against rate-limited endpoints.

MessagingE2ETests provides a representative end-to-end workflow. It The test spins up against the real containerized stack described in §6.4 — not a mocked server, an actual NexusTeam.Server talking to actual Oracle, MongoDB, and Redis containers — registers two distinct users through the real /auth/register endpoint, logs both in to obtain real JWTs, opens a real WebSocket connection for each, has one user create a chat and send a message through the real REST endpoint, and then asserts three separate things: that the REST call itself returned the expected message DTO, that the other user's WebSocket connection received a ChatMessageContract carrying matching content within a bounded timeout, and that a subsequent GET for message history returns the same message from persisted storage. A test built this way is slower than a unit test — it pays for a real server boot and real database round-trips — but it is also much harder to fool: a unit test that mocks the repository can pass even if the real MongoDB query is subtly wrong, while this test cannot pass unless persistence, the REST layer, and the WebSocket broadcast are all simultaneously correct. This is the reasoning behind keeping E2E coverage as a separate, deliberately-run-less-often suite (e2e-tests.yml) rather than trying to fold everything into the faster unit-test suite: the two suites are answering different questions, and collapsing them would weaken one to speed up the other.

Load Testing. Simulated at a modest scale appropriate to the project: multiple concurrent users, multiple active chats, simultaneous WebSocket connections, elevated message throughput, and repeated authentication requests, primarily to validate rate limiting and connection-manager correctness rather than to establish production-scale capacity numbers.

Error Handling Testing. Invalid requests, unavailable resources, authentication failures, and simulated database failures are checked against ExceptionHandlingMiddleware's mapping (§4.39), confirmed by ExceptionHandlingMiddlewareTests.

Logging Testing. Controlled events — authentication failures, invalid API requests, connection failures, unexpected exceptions — are verified to produce the expected diagnostic log entries via RequestLoggingMiddlewareTests, while confirming that sensitive information (passwords, tokens) is never written to logs.

Deployment Testing. The Docker-based environment is tested end to end: starting the Compose stack, starting databases and infrastructure, initializing database structures via the seeder, starting the backend, starting the web client, and verifying /health//health/ready//health/live. The docker-platform-ci.yml and docker-desktop-hosts.yml GitHub Actions workflows automate this on every push.

The platform coverage behind that automation is broader than a single "does it start" check. docker-platform-ci.yml runs whenever Docker, server, seeder, web, or configuration files change: it validates docker-compose.yaml itself, builds the server, seeder, and web images for both linux/amd64 and linux/arm64, starts and verifies the complete stack on native Linux x64 and Linux ARM64 GitHub-hosted runners, and — because a stack that only works on a pristine volume is not good enough — runs the seeder twice and performs a warm restart against already-populated volumes to catch bugs that only show up on a second startup. Windows and macOS are covered indirectly through the same two image architectures, since Docker Desktop on those platforms runs Linux images inside its own Linux VM. A second workflow, docker-desktop-hosts.yml, targets real Windows and macOS machines directly rather than emulated ones; it is nightly and can also be dispatched manually, but stays disabled until self-hosted runners carrying the right labels (Windows/X64, Windows/ARM64, macOS/X64, macOS/ARM64, each combined with docker-desktop) are registered and

the repository variable ENABLE_DOCKER_DESKTOP_RUNNERS is set — a deliberate design choice so that a missing runner produces a visibly-disabled workflow rather than a job that queues forever waiting for hardware that isn't there.

End-to-End Testing. tests/e2e/NexusTeam.E2E.Tests runs a representative complete workflow — E2E-01, Complete Messaging Workflow: start the system → register User A → register User B → login as User A → create a chat with User B → send a message → login as User B → verify the message and its delivery/read status — alongside dedicated suites for authentication, chat/folders, messaging, attachments (BinaryDataE2ETests), avatars, profile/preferences, WebSocket behavior, device lock, security, and the general API contract.

Example End-to-End Test Matrix.

Scenario	Suite	Key assertion
Register → login → refresh	*AuthenticationE2ETests*	Tokens issued and valid; duplicate registration rejected
Create chat → send/edit/delete message	*MessagingE2ETests*	Realtime delivery + persisted history match
Enable device lock → request from unknown device	*DeviceLockE2ETests*	401 outside the allow-listed routes
Upload attachment → download it	*BinaryDataE2ETests*	Byte-identical round trip, correct content type
Unauthorized chat access	*SecurityE2ETests*	401/403, no data leak

Defect Classification.

- **Critical** — the system cannot start, or a fundamental security/data-integrity problem exists (e.g. an authorization bypass).

- **High** — a major core feature such as authentication or messaging does not work.
- **Medium** — a secondary feature (e.g. translation, image generation) does not work correctly, but core communication remains available.
- **Low** — a cosmetic or minor usability issue.

This classification does more work than just labeling severity for a bug tracker — it directly determines how a defect gets caught, which loops back to the layered testing strategy discussed earlier in this chapter. A Critical-severity authorization bypass is exactly the kind of thing the E2E SecurityE2ETests suite exists to catch before it ever reaches a human tester, because waiting for a person to notice a data leak during manual testing is far too slow a feedback loop for something that severe. A Low-severity cosmetic issue, by contrast, is realistically only ever going to be caught by a person actually looking at the rendered UI — no unit test asserts that a button's padding looks right — which is precisely why UI/visual testing remains manual (§6.44) rather than automated: the fastest way to add automated coverage for cosmetic issues would be visual-regression screenshot testing, which is listed under future testing improvements below because it wasn't judged worth the setup cost for the severity of defect it would catch, given the project's timeline.

Defect Management. Each defect is documented with: description, reproduction steps, expected behavior, actual behavior, severity, and affected component, tracked as GitHub issues alongside the CI run that surfaced it where applicable. Recording the affected component alongside severity connects defect management directly back to the architecture in Chapter 3 — a defect tagged against WebSocketConnectionManager, for instance, is immediately known to sit in the realtime layer rather than the REST layer, which shortens the path from "a bug was reported" to "the right person is looking at the right code" in a project where responsibility is split across four people by architectural area (§7.3) rather than by feature. This is a small process detail, but it is the concrete

mechanism that makes the architecture chapter's separation of responsibilities pay off operationally rather than only existing on paper — a well-drawn architecture diagram is only useful for triage if defects are actually filed against the components it describes.

Automated Testing. Unlike the previous documentation revision, the project now contains a substantial automated test project rather than only application code.

```
tests/
├── unit/
│   ├── NexusTeam.Server.Tests/   (60 files: Configuration, Controllers,
│   │                              Services, Middleware, Data)
│   └── NexusTeam.Shared.Tests/
└── e2e/
    └── NexusTeam.E2E.Tests/      (12 files: Auth, Chat/Folders, Messaging,
                                   Attachments, Avatars, Profile/Preferences,
                                   WebSocket, DeviceLock, Security, API contract)
```

Figure 26 — Automated Test Suite Structure.

Both suites run in CI via unit-tests.yml and e2e-tests.yml, with docker-platform-ci.yml/docker-desktop-hosts.yml validating the containerized stack itself and a badge-publishing script (create-test-badge.py/publish-test-badge.sh) reporting pass/fail status.

The 60-file NexusTeam.Server. Tests project breaks down by what it exercises: 9 files test configuration-option validators (one per option class introduced in §3.27, plus supporting assertion helpers), 9 test controllers directly, 6 test middleware components, 18 — the largest single group — test services, and 8 test FluentValidation request validators. A separate NexusTeam.Shared.Tests project covers the shared library itself rather than assuming it's implicitly correct because the server and client tests pass: domain exceptions, helper utilities (file handling, ID generation, message-content processing, pagination, password hashing, Redis key formatting, the system clock abstraction), the JSON serializer factory, and value objects like the page-request DTO. Testing the shared library on its own matters precisely because it's shared — a bug there would silently affect the server and both clients at once, so it gets first-class test

coverage rather than being tested only indirectly through whichever consumer happens to exercise the buggy path.

Unit Testing Recommendations. The highest-value targets — already covered — are components with deterministic business logic: validators (*OptionsValidatorTests), authentication services, token services, message services, chat services, rate limiting, and middleware. Remaining gaps include broader coverage of the WPF client's ViewModels and services, and of the web client's app.js.

Integration Testing Recommendations. Recommended and largely implemented via the E2E suite: authentication against Oracle, message persistence against MongoDB, session/rate-limit operations against Redis, REST controllers against their services, and WebSocket connection establishment/broadcast — all exercised against the real containerized stack rather than mocks.

Testing Limitations. Client-side automated coverage is stronger on the backend than on either client: the WPF client and the web client are still tested mostly manually. Large-scale performance and load testing has not been run at production scale. Realtime call quality under adverse network conditions (packet loss, high latency, NAT traversal edge cases) needs additional testing, particularly once web-client calling is implemented. UI/visual and usability behavior is inherently hard to verify through backend or even E2E tests alone.

Future Testing Improvements.

- Expand automated UI testing for the web client (e.g. Playwright) and, where practical, the WPF client.
- Add load/performance testing at a larger simulated scale, particularly for WebSocket fan-out and MongoDB shard behavior.
- Extend call testing for web calls with additional media, screen-sharing, failure, and reconnection scenarios.

- Track test coverage over time via the CI badge and gate merges on it not regressing.

Testing and Requirements Traceability. Testing connects directly to Chapter 2's requirements — for example FR-10 (Real-Time Delivery) → WebSocketE2ETests; NFR-03 (Resource Authorization) → SecurityE2ETests; NFR-30 (Production Deployment) → docker-platform-ci.yml plus the manual deployment checklist in §8.17 — keeping testing anchored to the original requirements rather than drifting toward ad hoc coverage.

Overall Test Evaluation. Testing prioritizes authentication, messaging, realtime communication, data persistence, and security, because these features form the core of the application. Testing at multiple levels matters because NexusTeam spans several independent technologies: a passing API test does not guarantee a message renders correctly in either client, and a successful database write does not guarantee the corresponding WebSocket event reaches the other participant. Integration and end-to-end testing are therefore essential complements to unit testing, not optional extras.

Summary. Testing NexusTeam requires a multi-level approach across several interconnected applications and infrastructure components. Functional testing verifies authentication, device lock, users, chats, messages, attachments, folders, presence, calls (including call history), and additional features. API testing verifies the REST interface; WebSocket testing verifies realtime communication and call signaling. Database testing covers Oracle, MongoDB, and Redis. Client and UI testing verify the web and WPF clients from the user's perspective. Security testing covers authentication, authorization, device binding, token handling, rate limiting, and protected WebSocket connections.

Unlike the earlier revision of this document, the project now has a genuine automated test suite — 60 unit-test files in total (50 in NexusTeam.Server.Tests and 10 in NexusTeam.Shared.Tests) plus 10 E2E test classes across 12 E2E C# files including

infrastructure/assembly files. These run automatically in CI alongside the Docker-stack validation pipeline — giving the team much stronger regression protection than manual testing alone.

7. Project Management

Team Organization and Development Workflow

Project management coordinated development across backend, client implementation, UI design, testing, requirements/architecture, and documentation. Changes in one area regularly touched another: a backend API change could require client changes, and a shared WebSocket contract change could affect the server and both clients at once. The approach rested on clear responsibility distribution, version control, milestone-based development, regular coordination, and continuous integration of the individual components into one working system — culminating, this document, in an actual production deployment.

Team Organization. The team split along the project's major areas — requirements/architecture/documentation, backend and client implementation, UI, and testing — while developing collaboratively, since the components were not independent of one another.

The shared project also affects team coordination. Changes to shared DTOs and contracts can affect the server and both client applications, so changes to these definitions require coordination with the components that consume them. This makes cross-component dependencies visible during development and reduces the risk of incompatible duplicated definitions.

Responsibility Distribution.

- Requirements Management, Architecture, Documentation (Sofiia Khyzhnychenko) — defining functional/non-functional requirements, the system architecture and its diagrams, and keeping the project documentation current, including this document.

- Backend and Client Implementation (Vladyslav Zaplitniy) — the ASP.NET Core backend, the WPF client, and the shared project.
- User Interface (Anna Kornet) — the web client's and WPF client's interaction design and visual implementation.
- Testing (Halil Hakan Karabay) — the testing strategy, test cases, and (this document) the growing automated unit/E2E test suite.

Development Workflow. The process was iterative rather than a single linear pass: Requirement → Design/Architecture decision → Implementation → Test → Integrate → Document, repeated per feature rather than for the whole system at once.

Milestone-Based Development. Milestones provided intermediate targets: core authentication and chat/messaging first, then attachments/folders/presence, then extended features (translation, image generation, voice), then hardening (device lock, resource authorization, rate limiting — tracked individually in the project TODO.md), and finally the production deployment pipeline.

Planning and Task Breakdown. Large features were split into smaller technical tasks. Messaging, for example, broke down into: message model and MongoDB schema → REST endpoints (send/edit/delete/history) → WebSocket contracts and broadcast → client ViewModel/UI → tests.

Task Dependencies. Some tasks were independent; others depended on prior work — a new feature's UI could be designed before its API existed, but final integration depended on the API being available.

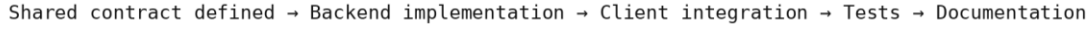


Figure 27 — Task Dependency Chain.

Version Control. Git was the primary version-control system, hosted on GitHub, providing a central location for source code, issue tracking, pull requests, and the GitHub Actions CI workflows described in Chapter 6.

Branching Strategy. Work happened on feature branches off main, merged via pull request once the relevant unit/E2E tests passed in CI.

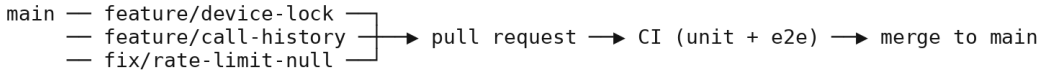


Figure 28 — Git Branching Workflow.

Commits. Commits represented meaningful, logically scoped changes rather than unrelated bundles — for example "Add device-lock middleware and PIN hashing" or "Fix null participant list validation (CascadeMode.Stop)" rather than a single catch-all commit per day.

Integration Process. A feature was considered integrated once its components could operate together end to end. Messaging integration, for example, required the MongoDB schema, the REST controller/service/repository, the WebSocket contract and broadcast, and both clients' UI to all agree on the same shape.

Shared Contracts and Team Coordination. NexusTeam.Shared was particularly important for coordination: without it, the server and each client would maintain independent copies of the same DTOs and WebSocket contracts, risking drift. A change to a shared type is felt immediately at compile time by every project that references it, which surfaced integration mismatches early rather than at runtime.

Communication Between Team Members. The most important recurring topics were API/contract changes, WebSocket event additions, database schema/collection changes,

and — once the deployment pipeline existed — coordination around what could safely go into the next production release.

Change, Risk, and Quality Management

Changes were evaluated by their blast radius: Proposed change → assess affected components (server / web / WPF / shared) → implement → update tests → update documentation → review → merge.

Issue and Defect Management. A problem is recorded with reproduction steps and severity (§6.39), assigned to the responsible area, fixed, verified by the relevant test suite, and then closed. Completed fixes are also recorded in project TODO.md. Examples include changes to WebSocket payload serialization, authorization, validation, device-lock behavior, and deployment-related issues.

Risk Management.

- **Integration risk** — a change in one component breaking another; mitigated by shared contracts and CI.
- **Infrastructure risk** — misconfigured Oracle/MongoDB/Redis/Docker breaking local or CI environments; mitigated by the seeded, containerized Compose stack used everywhere.
- **Deployment risk** — a production release failing on the VM; mitigated by the health-checked, auto-rolling-back deploy.sh (§8.17).

Technical Debt Management. Recognized items include: the demo-account seeder recreating fixed credentials on every startup (fine for a VPN-only demo, not for public exposure); differences between the web and WPF live-call implementations; and still-growing UI-side automated test coverage (Chapter 6). These are tracked explicitly rather than left implicit, alongside the resolved items already logged in the project TODO.md above — the distinction the team draws is between debt that has already been paid down

(the milestones listed above) and debt that is still open and simply needs to stay visible rather than be forgotten.

Project Quality **Control**.

Code review — changes to shared components were reviewed before merging. CI gates — pull requests had to pass the unit and E2E test workflows. Manual verification — new features were exercised manually against the Docker Compose stack before being considered done.

Definition of Done. A feature is done when: the requirement is understood and traced to an FR/NFR (Chapter 2); it's implemented per the architecture (Chapter 3); it has corresponding unit and/or E2E test coverage; it's reflected in the relevant client(s); and the documentation for it is updated in the same cycle (§8.23).

Project Management Across Specializations

Documentation was treated as an ongoing activity, not a final step: it covers requirements, architecture, implementation, UI, testing, project management, and deployment, and — as with this document — is expected to be updated whenever the codebase moves meaningfully ahead of what's written down.

Project Management and Architecture. Because different members worked in different areas, clear component boundaries (Chapter 3) were a project-management necessity, not just a technical nicety — they are what let backend, client, and UI work proceed in parallel without constant blocking.

Project Management and Testing. Testing was integrated into the workflow rather than deferred to the end: once a feature was implemented, its tests were added in the same cycle, which is why the current test suite (Chapter 6) tracks the feature set closely rather than trailing far behind it.

Project Management and Requirements. Requirements (Chapter 2) drove prioritization: core requirements (authentication, messaging, realtime communication, persistence) came first because the system needed them to function at all; extended features (translation, image generation, voice calls, device lock) followed once the core platform was stable.

Project Progress Tracking. Progress was tracked through requirements completed, milestones reached, tests passing in CI, and — toward the end of the project — successful deployments to the production VM, rather than through time-based estimates alone.

Collaboration Between Specializations.

Requirements ↔ Architecture — requirements determined which architectural capabilities were necessary (§2.22). Architecture ↔ Implementation — the layered design in Chapter 3 shaped how Vladyslav structured the backend and clients. Implementation ↔ Testing — new services and controllers arrived with a natural set of test targets. All ↔ Documentation — every area's work is reflected in this document.

Challenges, Lessons, and Summary

Coordinating changes across multiple components was the first challenge — a small backend change could ripple into both clients. The second was integrating several infrastructure technologies, each with its own configuration and operational quirks (Oracle, a sharded MongoDB cluster, Redis, Docker). The third, newer challenge was operating a real production deployment: secrets, certificates, and a VPN-restricted VM introduce operational concerns a local Docker Compose stack does not have.

A continuing project challenge was keeping the documentation aligned with the implementation as the system changed. Earlier documentation no longer reflected parts of the client architecture, testing status, and production deployment. The team therefore

treated documentation updates as part of the development process rather than as a final step.

Lessons Learned.

Clear interfaces are essential — shared contracts significantly reduced misunderstandings between developers. Tests catch integration breakage early — once the automated suite existed, contract mismatches surfaced in CI rather than during manual testing. Deployment should be scripted and health-checked from day one — doing this later meant retrofitting rollback safety rather than having it from the start. Documentation needs an owner and a checkpoint, not just good intentions — as the paragraph above describes, "someone will keep it updated" without a specific someone and a specific moment to do it reliably doesn't happen on its own.

Future Project Management Improvements.

- Extend CI to gate on client-side (not just backend) test coverage.
- Formalize a lightweight release-notes process tied to each production deployment.
- Rotate a documentation-owner role at each milestone so documentation updates remain aligned with implementation.

Project Management Summary. Development required coordinating requirements/architecture, backend/client implementation, UI, and testing across four team members. Git, feature branches, CI, milestones, and shared contracts supported collaborative development, and the project has since grown to include a scripted, health-checked production deployment process.

Final Project Management Perspective. NexusTeam demonstrates that successful software development depends not only on implementing individual features but on coordinating the relationships between them. Requirements management, architecture,

implementation, UI development, testing, documentation, and — now — production operations were combined into one development process; the division of responsibilities let each member focus on a specialized area while shared architecture, common contracts, version control, and integration activities kept the project consistent.

8. Documentation

Documentation Purpose and Structure

Documentation matters for NexusTeam because the system spans multiple applications, three databases, several communication protocols, and containerized infrastructure that now includes a real production deployment. The goal is not only to describe the final implementation but to make the system understandable to developers, testers, administrators, and evaluators. A developer joining the project should be able to understand the architecture, install the infrastructure, start the applications, identify the main components, and continue development without reverse-engineering the system from source. Documentation covers project overview, requirements, architecture, implementation, UI, testing, project management, and — this document's main addition — an accurate account of production deployment.

Documentation Objectives. Explain the purpose and scope of NexusTeam; describe the functional and non-functional requirements; explain the system architecture with diagrams that match the current codebase; document important implementation decisions; and describe how the system is actually deployed and operated, not only how it runs locally.

Documentation Structure. The documentation is divided into the nine chapters listed in §1.10, corresponding to the major aspects required for project evaluation, plus the supporting Markdown documents kept alongside the source code (docs/ARCHITECTURE.md, docs/CLIENT.md, docs/SERVER.md, docs/SECURITY.md, docs/INSTALLATION.md, docs/Docker.md, docs/DOCKER_CI.md, and DEPLOYMENT.md), which this chapter is anchored to.

Documentation for Different Stakeholders.

End users need to know how to access the system and log in — covered by the README's quick-start section (open http://localhost:8080, or the production URL, with the documented demo accounts). Developers/administrators need architecture, installation, security, server, client, Docker, and deployment documentation (§8.7–§8.17). Evaluators need the requirements-to-architecture-to-testing traceability established across Chapters 2, 3, and 6.

These three audiences don't just need different documents — they need different starting points into the same body of knowledge, which is why the documentation is organized as a set of focused files (README.md, docs/ARCHITECTURE.md, docs/CLIENT.md, docs/SERVER.md, docs/SECURITY.md, docs/INSTALLATION.md, DEPLOYMENT.md, and this chapter-based academic document) rather than one undifferentiated wall of text. An end user reading README.md never needs to know that MongoDB runs as a three-shard cluster; a new backend contributor reading docs/SERVER.md needs exactly that detail within the first few paragraphs; and an evaluator reading this document needs both, connected explicitly back to the requirements and tests that justify them, which is the specific job this academic documentation set does that the more implementation-focused docs/ files don't attempt.

Project Overview Documentation. Covers the purpose of the application, target users, main functionality, technology stack, and scope — Chapter 1 of this document.

Requirements Documentation. Covers functional requirements, non-functional requirements, and stakeholders — Chapter 2 of this document, including the requirements added or corrected this document (device lock, call history, production deployment).

Technical Documentation: Architecture Through Database

Explains how the major components interact: client-server architecture, backend layers, the web client's plain-JS architecture, the WPF client's MVVM architecture,

polyglot persistence, WebSocket architecture, and production deployment architecture — Chapter 3, cross-referenced with docs/ARCHITECTURE.md.

Implementation Documentation. Provides more technical detail than the architecture chapter: the structure and responsibilities of controllers, services, repositories, middleware, and client components — Chapter 4, cross-referenced with docs/SERVER.md and docs/CLIENT.md.

API Documentation. Describes available REST endpoints, HTTP methods, required authentication, and request/response shapes (§3.13, §4). The server also exposes Swagger/OpenAPI documentation in development for interactive exploration. The Swagger document is generated from the controllers and DTOs themselves — action attributes, XML doc comments, and the shared request/response types in NexusTeam.Shared — rather than hand-maintained separately, so it cannot drift from the actual endpoint signatures the way a hand-written API reference sometimes does; its main current gap, tracked in §8.28, is that it documents shape well but doesn't always explain why an endpoint behaves the way it does (for instance, why a folder read for a non-owner returns 404 rather than 401 — a deliberate information-disclosure decision recorded in the project TODO.md rather than in the Swagger description itself).

WebSocket Documentation. Describes the /ws connection endpoint, the authentication process (JWT at connection time, with a limited grace period — §4.23), the shared message envelope (§3.15), and the full set of contract types, including the call-signaling and device-lock-adjacent contracts added since the previous revision.

Database Documentation. Distinguishes Oracle (structured user-related data), MongoDB (document-oriented communication data, deployed as a sharded cluster), and Redis (fast-access/temporary data) — §3.9–§3.12 and Figure 3.

Database Seeder Documentation. NexusTeam.DbSeeder runs once per environment startup (development and, currently, production alike) to initialize

Oracle/MongoDB/Redis structures and seed four demo accounts with a shared password — explicitly documented as unsuitable for exposure to untrusted users without being disabled or reconfigured first (§3.35, §4.41).

Environment, Configuration, and Deployment Documentation

The typical process, per docs/INSTALLATION.md and the root README.md: install the prerequisites (Docker Desktop on Windows/macOS or Docker Engine + Compose v2 on Linux, with at least 8 GB RAM allocated to Docker; the .NET 8 SDK if building outside containers; Git); clone the repository; copy .env.example to .env (and, for the WPF client, restore NuGet packages); and run docker compose up -d --build from Nexus-Team/. Verification follows the same pattern documented for troubleshooting below: docker compose ps should show every service healthy, the seeder container should have exited 0 (it is one-shot, not long-running), and http://localhost:8080 should serve the web client. Optional tooling — mongosh, redis-cli, SQL*Plus/Oracle SQL Developer — is documented for administrators who need to inspect a database directly rather than through the application.

Development Environment. Requirements: .NET 8 SDK/runtime, a C#-capable IDE, Docker Engine/Docker Desktop with Compose v2, and (for the WPF client) Windows.

Configuration Documentation. Configuration is separated from application logic via dedicated option classes (§3.27) for Oracle, MongoDB, Redis, JWT, BCrypt, CORS, rate limiting, and device lock, bound from appsettings.json in development and from environment variables/.env.production in production. The development appsettings.json ships pre-configured for the local Docker Compose stack — Oracle at localhost:1530/FREEPDB1, MongoDB at localhost:27018, Redis at localhost:6380 — specifically so that a contributor never has to edit it just to get the application running locally. Production instead overrides the same settings through environment variables (ORACLE_CONNECTION_STRING, MONGODB_CONNECTION_STRING,

REDIS_CONNECTION_STRING, JWT_SECRET_KEY, and the other sensitive values deploy.sh refuses to leave at their development defaults, per §2.19 and §8.17), which keeps the one appsettings.json file safe to commit to source control while still letting every environment-specific secret live outside it.

Docker Documentation. docker-compose.yaml defines the server, web client, Oracle, the MongoDB sharded cluster (config server, two shards, mongos router), Redis, and the one-shot database seeder, plus — in the production compose configuration — the Nginx/Certbot TLS gateway and the coturn TURN relay (deploy/gateway/). Network configuration uses a bridge network (nexusteam_network) with internal DNS resolution between containers and named volumes for persistent data, per docs/Docker.md and docs/DOCKER_CI.md (which additionally documents the docker-platform-ci.yml/docker-desktop-hosts.yml CI workflows that validate the stack on every push).

Deployment Documentation. Deployment documentation now covers two environments rather than one.

Development — docker compose up -d --build from Nexus-Team/, exposing the web client on http://localhost:8080.

Production— driven by deploy.sh, per DEPLOYMENT.md:

1. Prerequisites: a Linux server reachable through the team VPN, SSH public-key auth, Docker Engine + Compose v2, ≥8 GB RAM, and a deployment directory (/opt/nexus-team) owned by the deploy user.
2. Production secrets live in a local, git-ignored .env.production (independent, randomly generated secrets plus non-default Oracle passwords — the script refuses CHANGE_ME/development values); on first deploy it’s uploaded to /opt/nexus-team/shared/.env with mode 0600.
3. Running DEPLOY_HOST=… DEPLOY_USER=… DEPLOY_SSH_KEY=… ./deploy.sh validates local tooling, Git state, VPN/SSH connectivity, Docker

access, and secrets; archives the selected committed revision; uploads it to a new timestamped release directory; builds new images alongside the still-running old stack; stops the old stack with docker compose down --remove-orphans --timeout 120 (deliberately without --volumes, so Oracle/MongoDB/Redis data survives); starts the new stack; and waits for nexusteam_server and nexusteam_web to report healthy (§4.26).

4. Once healthy, the production gateway completes an HTTP-01 challenge and Certbot obtains a trusted, short-lived (six-day) IP certificate; Nginx activates HTTPS; a renewal sidecar checks every six hours.
5. On success, current is repointed to the new release and old releases beyond the retention count (DEPLOY_KEEP_RELEASES, default 3) are pruned. On failure, the failed stack is stopped and the previous release is rebuilt and restarted automatically.
6. Only the TLS gateway (443, with 80 redirecting/serving ACME challenges) and the coturn TURN relay ports (3478, 5349, 49160–49200) are public; the API (5251), Oracle (1530), MongoDB (27018), and Redis (6380) are bound to 127.0.0.1.
7. Operationally: ssh deploy@<host>, cd /opt/nexus-team/current, docker compose ps / docker compose logs --follow web server.

Troubleshooting Documentation.

- **Backend does not start** — possible causes include an unavailable database (check docker compose ps and the seeder's completion), a port conflict, or a missing/invalid configuration value.
- **Deployment health check never turns green** — check docker compose logs server web on the VM for the actual startup error before assuming the

deployment script itself is at fault; deploy.sh prints these logs automatically on failure.

- **Login works locally but not after a redeploy** — usually a symptom of DEPLOY_ENV_FILE being left unset when it should have been empty (reusing server-side secrets), producing a fresh JWT signing key that invalidates every previously issued token; the fix is operational (redeploy with the correct secrets handling), not a code change.
- **A container is healthy but the feature it backs doesn't work** — check the container's own logs before the application logs; a healthy container only means its health-check endpoint responded, not that every downstream dependency it needs (e.g. the MongoDB shard router) is actually reachable from inside it.
- **WPF client cannot connect to the server** — verify the Docker services are actually running (docker compose ps), that the server itself is up and connected to them, and that the IP/port passed on the command line match where the server is actually listening; a firewall blocking the port produces the same symptom as the server being down, so both need to be checked.
- **WPF client sends nothing / messages never arrive** — check the WebSocket connection state first, then the stored authentication token's validity, then the server logs; this is one of the few places where the fix could be on either side of the connection, so the ordering (client connection → client token → server) is chosen to rule out the cheapest explanation first.
- **WPF client crashes on startup** — check the client's own log files under %LocalAppData%\NexusTeam\Logs\, confirm the .NET 8 runtime is actually installed (a dotnet build on a machine with only the SDK doesn't guarantee the runtime the published executable needs is present), and confirm the machine meets the client's minimum requirements.

- **A previously logged-in session doesn't restore** — this points at the LiteDB-backed credential store (§4.31) rather than the server; clearing the stored credentials and logging in again is the practical fix, and checking token validity separately confirms whether the stored token had simply expired rather than being corrupted.

Logging Documentation. Logs are generated via Serilog: to console in development, to file in production. Documentation explains which events are logged (requests/responses, WebSocket connection lifecycle, authentication failures, exceptions), how log levels are configured, and confirms that sensitive values (passwords, tokens) are never written to logs (§4.40, §6.35).

Security, Maintenance, and Documentation Quality

Covers password hashing (BCrypt), JWT authentication and refresh tokens, resource-level authorization, device lock, rate limiting, security headers, CORS, and the production TLS/Let's Encrypt setup — Chapter 3 §3.29 and docs/SECURITY.md, which additionally documents WebSocket security, voice-call security, security hardening already implemented (see the project TODO.md), current limitations, and the incident-response/vulnerability-reporting process.

Maintenance Documentation. Helps a developer answer: where is a feature implemented (Chapter 4, cross-referenced against §3.6–§3.8's layer descriptions), which database stores the related data (§3.9–§3.12), and which API endpoint or WebSocket contract is responsible (§3.13–§3.18, §4.18).

Adding a New Feature. Define requirement (Chapter 2) → update architecture if necessary (Chapter 3) → add/extend a shared contract if the feature crosses the client/server boundary → implement backend (controller/service/repository) → implement client(s) → add unit/E2E tests (Chapter 6) → update this documentation (§7.19, §7.20) → deploy (§8.17).

Documentation and Version Control. Documentation is versioned alongside the source code so implementation and documentation changes share the same project history; a new API endpoint or a new WebSocket contract should be documented in the same development cycle it's introduced, which is the standard this document was written to restore.

Documentation Quality.

Accuracy — information must correspond to the actual implementation; this document corrects client capabilities, API routes, controller/service/repository inventories, testing counts, and call architecture. Completeness — the major components and workflows are covered in the text, while the existing diagrams are intentionally preserved unchanged and will be synchronized separately.

API and UI Documentation Relationship. The same workflow can be documented at three levels. Sending a message, for example: User level — the user types text and taps send. Client level — the ViewModel/app.js calls the messaging service, which sends a REST request and/or emits a WebSocket contract. Server level — MessagesController validates and authorizes the request, MessageService applies business logic, MongoMessageRepository persists it, and WebSocketConnectionManager broadcasts the result.

Testing Documentation. Testing documentation records testing objectives, the test environment, test cases, and — since the previous revision — the actual automated test suite and CI pipelines described in Chapter 6, rather than only a plan for tests that don't yet exist.

Documentation and Future Developers. A future developer should be able to use this documentation to understand the project's purpose, set up the development environment, start the infrastructure, run the server and both clients, understand the production deployment, and locate the code responsible for any given feature.

9. Conclusion and Future Work

Conclusion and Achievements

NexusTeam was developed as a multi-platform communication system combining real-time messaging, user management, file sharing, chat organization, voice communication, AI image generation, and — as of this document — a real production deployment, all in one application. The project demonstrates the complete software development process: requirements analysis, architectural planning, implementation, UI development, testing, project management, and documentation, followed by shipping the system to a virtual machine reachable outside the development environment.

The resulting system has several interconnected components. The ASP.NET Core 8 backend provides the central application logic and communication interface. The web client — a responsive, framework-free HTML/CSS/JavaScript application served by Nginx — is the primary, universal way to reach the product; the optional WPF client provides a native Windows experience, while the web client provides browser WebRTC audio/video calls and screen sharing. The WPF client provides live audio through NAudio and WebSocket call communication.

NexusTeam is complete for the current software engineering project scope: the architecture described in this document is implemented in the codebase, the main functionality is covered by automated tests, and the system runs in a deployed environment. It is not a finished commercial product. Further work would still be required for broader automated client testing, larger-scale performance validation, additional security features, stronger observability, and other production-oriented requirements.

A central architectural characteristic is separation of responsibilities between layers: controllers provide the API interface, services implement business logic, repositories abstract database access, and middleware handles authentication, logging, validation,

rate limiting, device-lock enforcement, and error handling. The system uses three persistence technologies matched to the type of data being processed — Oracle for structured user data, a sharded MongoDB cluster for communication documents, and Redis for fast, temporary state.

Realtime communication runs through WebSockets, enabling messaging, typing indicators, delivery/read status, presence, and voice-call signaling, standardized through shared DTOs and WebSocket contracts. From a requirements perspective, the project addresses the main needs of a modern communication platform: users can authenticate (optionally with device-bound PIN protection), manage conversations, exchange messages, share files, organize chats, generate images, translate content, and use voice communication.

Testing now spans functional, API, database, WebSocket, security, client, integration, and end-to-end levels, backed by 60 unit-test files in total (50 backend/server and 10 shared-library) and 10 E2E test classes across 12 E2E C# files, running automatically in CI alongside Docker-stack validation.

Achievement of Project Objectives. The project meets the objectives set in Chapter 2: secure authentication (now with optional device binding); private and group communication; real-time message exchange with editing, deletion, reactions, forwarding, and status tracking; file and image attachments; user presence; chat-folder organization; voice messages on both clients; live audio/video calls and screen sharing on the web client; live audio calls on the WPF client; server-side call signaling and call history; AI image generation; WPF translation; and a scripted, health-checked, rollback-capable path to production deployment. The remaining gaps are tracked explicitly as future work below.

Technical Achievements.

Modular backend — controllers, services, repositories, middleware, and infrastructure components are cleanly separated, making the system easier to maintain and extend, as demonstrated by how cleanly device lock and call history were added on top of the existing layering. **Two clients, one backend** — the system shows how very different client technologies (framework-free JS and WPF/MVVM) can share one backend without duplicating business logic. Realtime communication — the WebSocket implementation supports messaging, presence, and call signaling well beyond ordinary REST. **Polyglot persistence** — Oracle, MongoDB, and Redis are each used for what they're good at rather than forcing everything into one store. A genuine test suite and CI pipeline — the project moved from "extensive application code with limited automated testing" to 60+12 automated test files running in CI on every push. A real production deployment — deploy.sh takes a committed revision to a VPN-restricted VM with health-checked cutover, automatic rollback, and Let's Encrypt-backed TLS, rather than stopping at "runs in Docker on a laptop."

Future Work

Several areas remain for further development.

Client-Side Automated Testing. Backend automated testing is now strong; client-side testing is the clearest remaining gap. Future work could add automated UI testing for the web client (e.g. Playwright, driving the same responsive breakpoints described in §5's Extended Features discussion) and expand automated coverage of the WPF client's ViewModels and services — the latter is more tractable than it might sound, since MVVM already isolates most logic (command handling, state transitions) from the actual XAML rendering, so a ViewModel can be unit-tested by instantiating it against mocked services without needing a running WPF window at all.

Continuous Deployment. The deployment pipeline (deploy.sh) is currently invoked manually. A future improvement would trigger it automatically from CI after tests pass

on main, turning the current CI/CD-adjacent setup into full continuous deployment. This is a smaller step than it sounds given the existing pieces: the script already validates secrets, builds images, performs a health-checked cutover, and rolls back automatically on failure (§8.17), so wiring it to a CI job mainly means deciding on a gating policy (every merge to main, or a manually-approved release step) rather than building new deployment machinery.

Improved Security. Even with the hardening already completed (resource-level authorization, device lock, rate limiting, suppressed server-identification headers — see the project TODO.md), further improvements are possible: centralized secret management beyond .env.production (a proper secrets manager rather than a file on the VM), additional authorization policies (moving from the current participant/ownership model toward role-based access control for group chats, for example, so a group could have moderators with different capabilities than ordinary members), security auditing, vulnerability scanning, and penetration testing.

Scalability and Performance. The architecture provides a foundation for scaling (a sharded MongoDB cluster, stateless JWT-authenticated servers), and §3's scalability discussion lays out concretely where that foundation would extend — multiple server instances behind a load balancer, Redis Pub/Sub for cross-instance WebSocket broadcast, Oracle read replicas — but large-scale load testing, WebSocket connection benchmarking, database performance optimization, Redis optimization, and production resource monitoring all remain future work rather than validated capacity.

Complete Web Client Calling. Web-client live calling is already implemented: browser WebRTC provides audio/video, ICE handling, and screen sharing, with WebSocket signaling and coturn TURN support. Future work is therefore focused on richer call quality diagnostics, adaptive media behavior, improved audio quality, and potentially further media features such as video-call enhancements rather than implementing the basic calling stack.

Push Notifications. New messages, missed calls, mentions, group activity, and account events could trigger push notifications when a user isn't actively viewing the application, improving usability on both desktop and web. The call-history subsystem's unseen-missed-call tracking (§4.26b) is effectively a small, already-built piece of this larger feature — the same "unseen event" concept would generalize to unseen messages and mentions.

Mobile Client. The backend's centralized API and realtime protocol are already positioned to support Android/iOS clients reusing the same shared contracts, without a backend redesign — a mobile client would consume the same REST endpoints (§4's endpoint table) and the same WebSocket envelope (§3.15) that both existing clients already use, making it primarily a new presentation layer over an interface that has already been proven out by two very different clients.

Improved UI and Accessibility. A shared cross-platform design system, further responsive layouts, keyboard navigation and screen-reader support, more extensive usability testing, and a web-client calling UI to match the WPF client's (§9.4.5, §5's UI Evaluation discussion).

Advanced Search. Basic message search within a conversation is already implemented through MessagesController. Future work can extend this into advanced search with filtering by user/date/attachment, conversation-wide or cross-conversation search, and file-type filtering.

End-to-End Encryption. A substantial future architectural change: messages encrypted client-side before leaving the sender and decrypted only by authorized recipients, requiring key generation, key exchange, key storage, device management (which the new device-lock feature is a natural building block for, since it already establishes a notion of "this specific device belongs to this user"), group encryption, and message synchronization across devices. This is deliberately listed last among the future-work

items because it is the one that would touch the most existing architecture — server-side features like translation and search that currently operate on plaintext message content would need to be reconsidered or explicitly scoped out once messages are opaque to the server by design.

Final Evaluation

The separation between clients, backend services, repositories, databases, and shared contracts provides a structure for further development. Features such as device lock, call history, rate limiting, health checks, and production deployment were added within this structure by following the same general process: requirement, architecture decision, implementation, testing, and documentation.

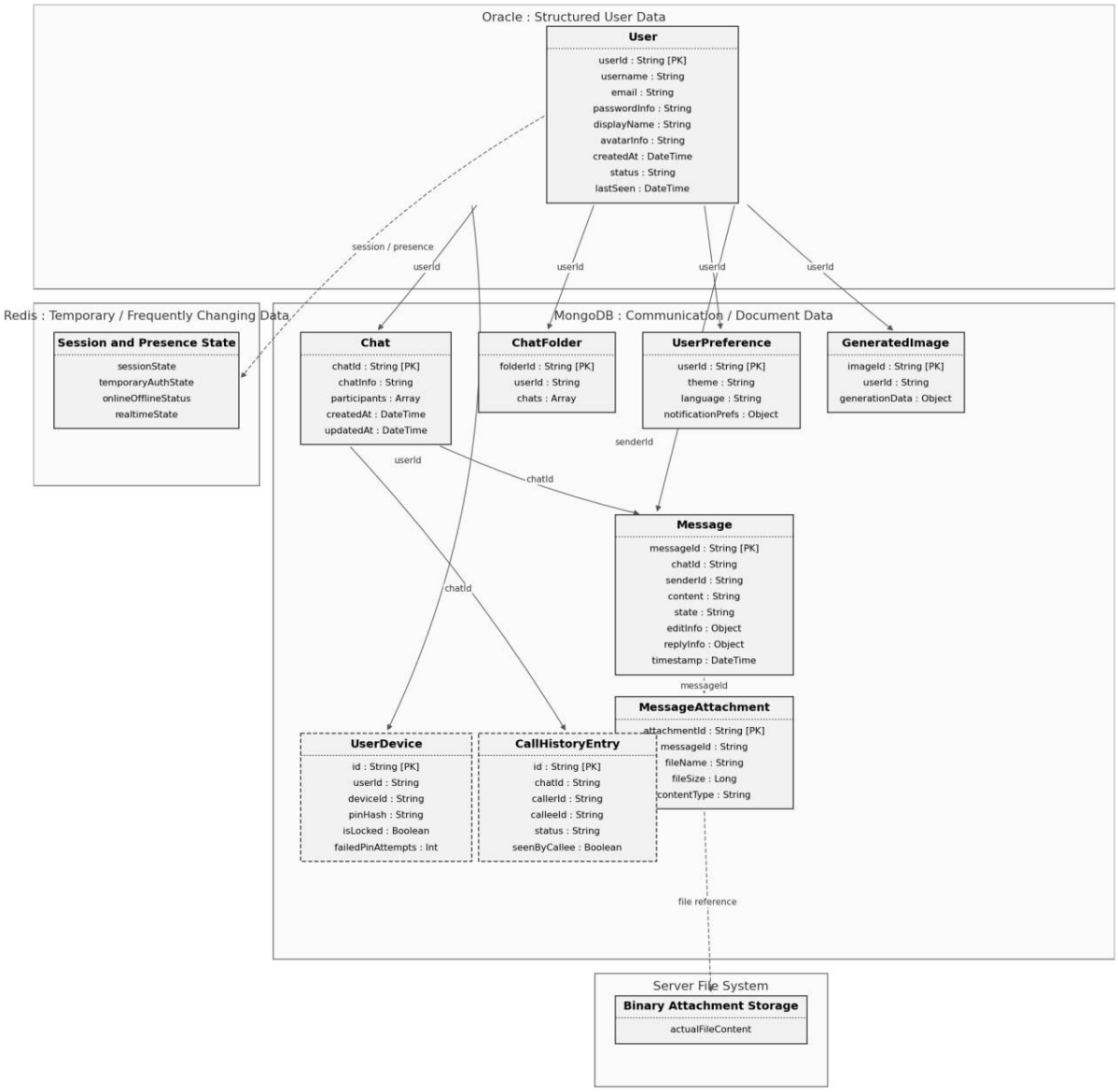
Final Evaluation. NexusTeam demonstrates the development of a complete software system rather than an isolated component: requirements engineering, software architecture, backend development, client development, UI design, three database technologies, realtime communication, automated testing, project management, and — now — production deployment operations are all represented in the delivered project.

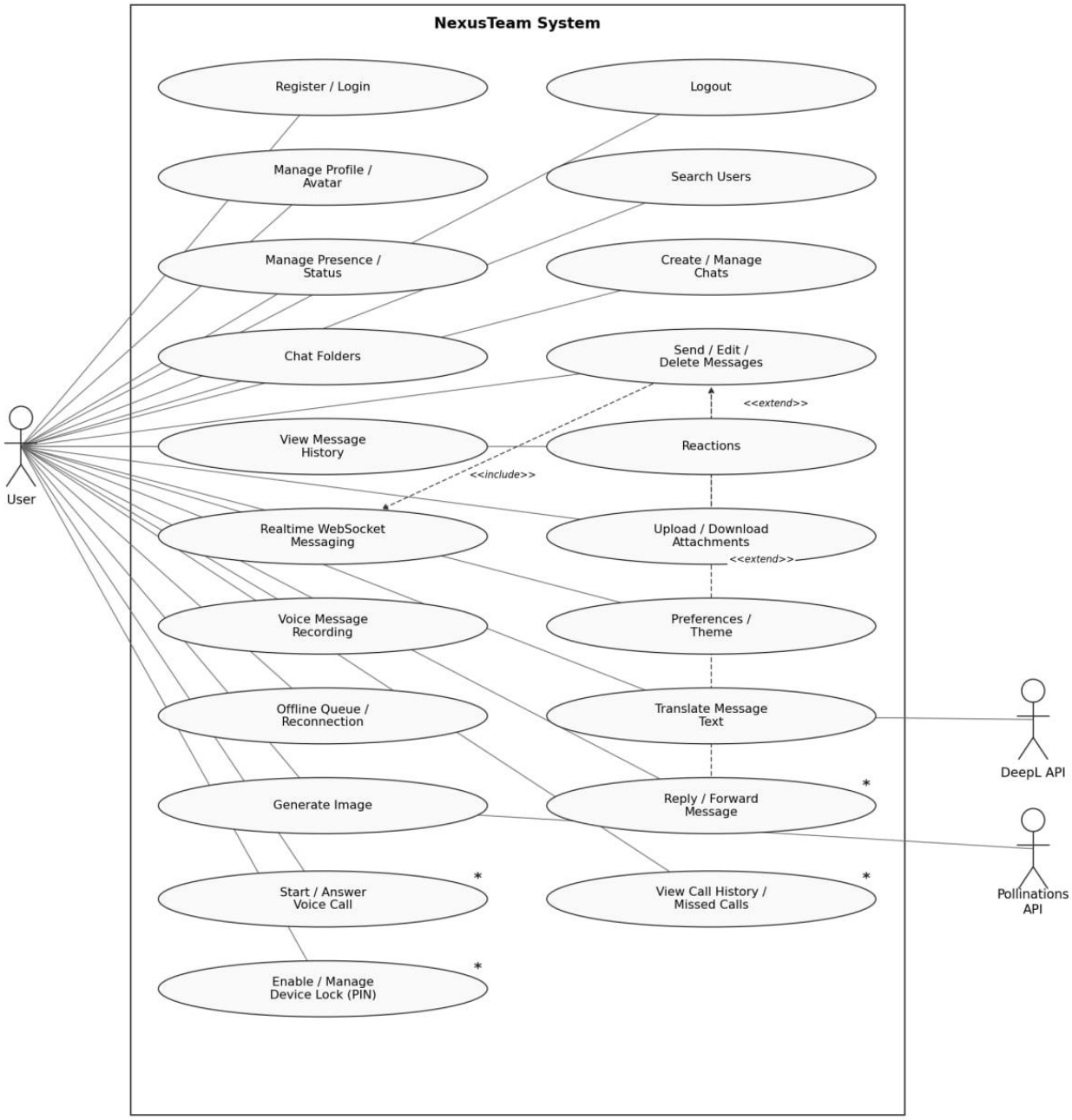
Final Conclusion. NexusTeam successfully establishes a modern multi-platform communication system, now running on a production virtual machine rather than only on development machines. The implementation provides the essential functionality for user authentication (with optional device binding), chat management, realtime messaging, attachments, user presence, organization, voice communication, AI image generation, and translation. The combination of ASP.NET Core, a framework-free responsive web client, an MVVM WPF client, WebSockets, Oracle, a sharded MongoDB cluster, Redis, Docker, and a scripted VM deployment demonstrates the integration of many technologies into one coherent, operable system.

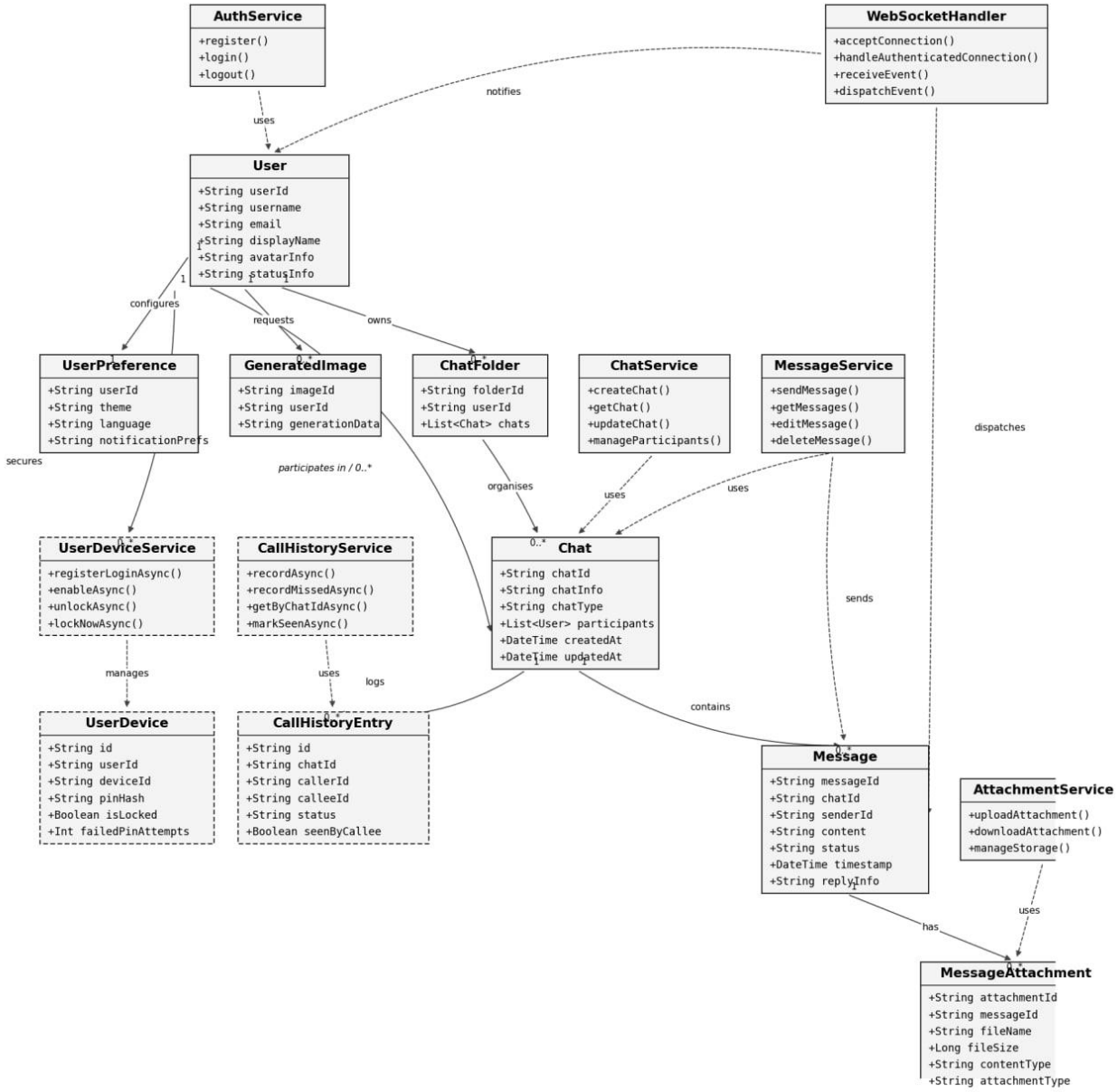
The modular architecture and separation of responsibilities provide a foundation for maintenance and further development. Current limitations include limited automated

client testing, the lack of large-scale performance validation, further security-hardening work, and opportunities to extend search and other advanced features. Overall, NexusTeam demonstrates a functional communication platform together with the software engineering practices used to design, implement, test, deploy, and document it.

Appendix A — Diagrams




NexusTeam System
Register / Login
Logout
Manage Profile / Avatar
Search Users
Manage Presence / Status
Create / Manage Chats
Chat Folders
Send / Edit / Delete Messages
<<extend>>
View Message History
Reactions
<<include>>
Realtime WebSocket Messaging
Upload / Download Attachments
<<extend>>
Voice Message Recording
Preferences / Theme
Offline Queue / Reconnection
Translate Message Text
Generate Image
Reply / Forward Message
Start / Answer Voice Call
View Call History / Missed Calls
Enable / Manage Device Lock (PIN)
User
DeepL API
Pollinations API


AuthService
+register()
+login()
+logout()
WebSocketHandler
+acceptConnection()
+handleAuthenticatedConnection()
+receiveEvent()
+dispatchEvent()
notifies
uses
User
+String userId
+String username
+String email
+String displayName
+String avatarInfo
+String statusInfo
configures
requests
owns
UserPreference
+String userId
+String theme
+String language
+String notificationPrefs
GeneratedImage
+String imageId
+String userId
+String generationData
ChatFolder
+String folderId
+String userId
+List<Chat> chats
ChatService
+createChat()
+getChat()
+updateChat()
+manageParticipants()
MessageService
+sendMessage()
+getMessages()
+editMessage()
+deleteMessage()
dispatches
secures
participates in / 0..*
organises
uses
uses
UserDeviceService
+registerLoginAsync()
+enableAsync()
+unlockAsync()
+lockNowAsync()
CallHistoryService
+recordAsync()
+recordMissedAsync()
+getByChatIdAsync()
+markSeenAsync()
Chat
+String chatId
+String chatInfo
+String chatType
+List<User> participants
+DateTime createdAt
+DateTime updatedAt
sends
manages
uses
logs
contains
UserDevice
+String id
+String userId
+String deviceId
+String pinHash
+Boolean isLocked
+Int failedPinAttempts
CallHistoryEntry
+String id
+String chatId
+String callerId
+String calleeId
+String status
+Boolean seenByCallee
Message
+String messageId
+String chatId
+String senderId
+String content
+String status
+DateTime timestamp
+String replyInfo
AttachmentService
+uploadAttachment()
+downloadAttachment()
+manageStorage()
uses
has
MessageAttachment
+String attachmentId
+String messageId
+String fileName
+Long fileSize
+String contentType
+String attachmentType